

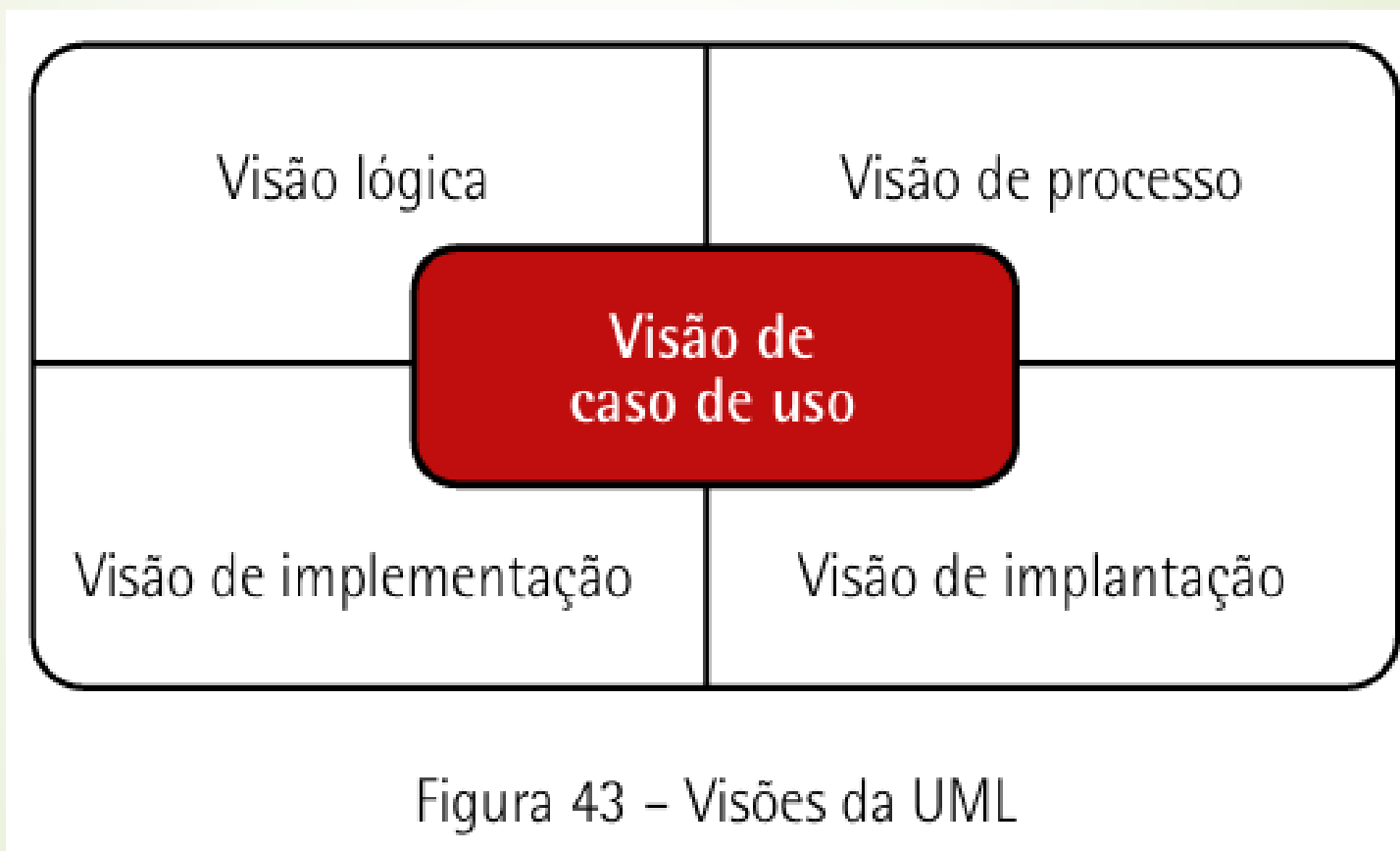


As 5 Visões da UML

APS – Análise e Projetos de Sistemas

Prof. Francisco Martins

As 5 visões propostas por Kruchten (1995)



1-Visão de caso de uso

- Tem como objetivo capturar as funcionalidades, os requisitos e seu comportamento sob a ótica do usuário final, ou dos atores.
- A visão de caso de uso é centralizada, uma vez que o que é produzido nessa visão é a base para as outras visões do sistema.

2-Visão lógica

- Tem como objetivo representar como as funcionalidades serão implementadas sob o aspecto da solução de projeto.
- Representa a estrutura estática de um sistema, seus componentes e o relacionamento entre eles e como esses interagem para resolver um determinado problema. Essa interação é capturada pela estrutura dinâmica do sistema.

3-Visão de processo

- Tem como objetivo capturar aspectos de paralelismo de execução de atividades sob o ponto de vista não funcional de um sistema.
- Representa uma visão mais técnica do sistema, na qual o trata como unidades de processos e processamentos, que podem ocorrer de forma síncrona ou paralela. Essas unidades também podem ser interpretadas como subsistemas.

4-Visão de implementação

- Tem como objetivo representar aspectos físicos necessários para a construção do sistema e como eles interagem e fazem interface com o sistema.
- São exemplos desses aspectos físicos: sistemas de software, programas e rotinas internas, banco de dados e bibliotecas utilizadas.

5-Visão de implantação

- Tem como objetivo representar a organização física de hardware do sistema, como computadores, servidores e periféricos e como eles se relacionam com o sistema.
- Essa visão é utilizada principalmente no processo de implantação, também chamado de instalação ou distribuição do sistema.

VISÃO	DIAGRAMAS
Visão de caso de uso	Diagrama de caso de uso Diagrama de processo Diagrama de atividade
Visão lógica	– Estrutura estática: - Diagrama de classe - Diagrama de objeto – Estrutura dinâmica: - Diagrama de estado - Diagrama de sequência - Diagrama de colaboração - Diagrama de interação - Diagrama de atividade
Visão de processo	São utilizados os mesmos diagramas utilizados na visão lógica, mas com ênfase na linha de execução do sistema
Visão de implementação	Diagrama de componente
Visão de implantação	Diagrama de implantação

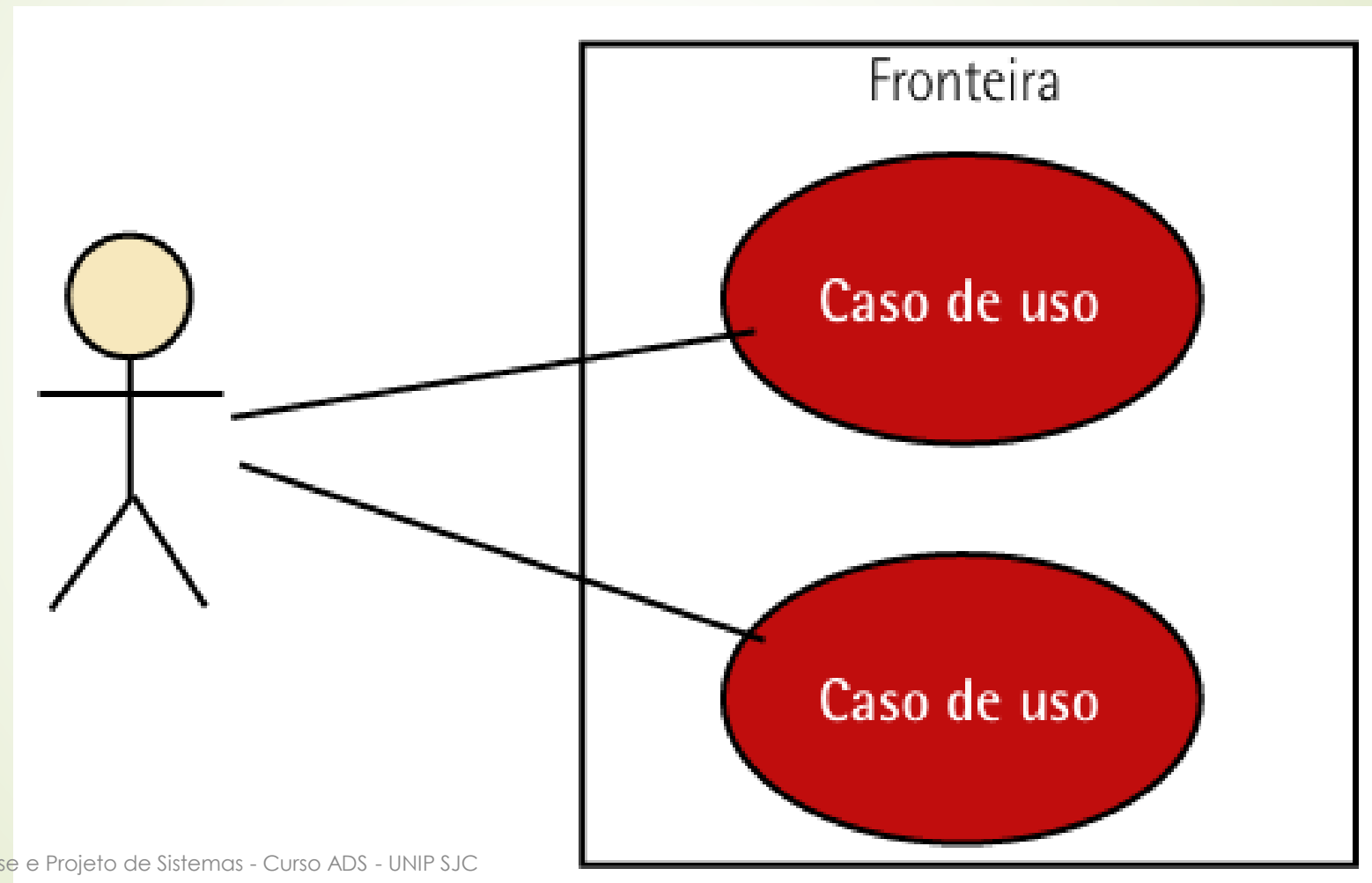


Orientação a Objetos Caso de Uso

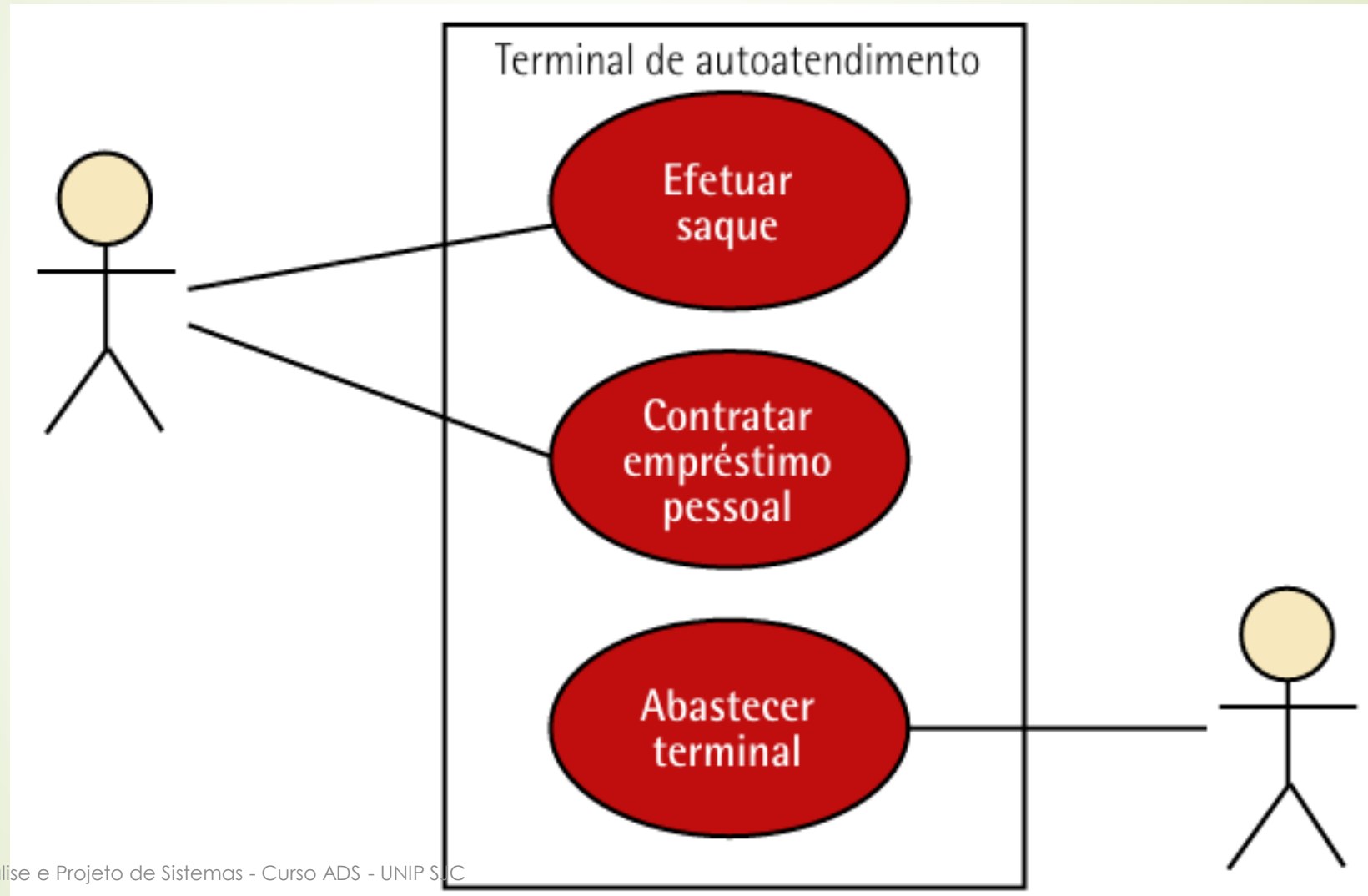
APS – Análise e Projetos de Sistemas

Prof. Francisco Martins

Diagrama de Caso de Uso

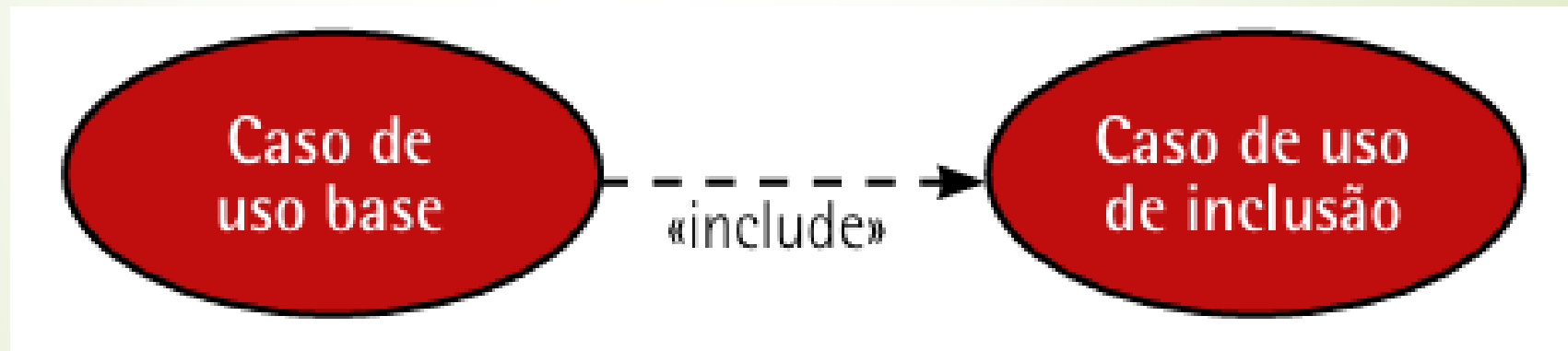


DCU - Exemplo



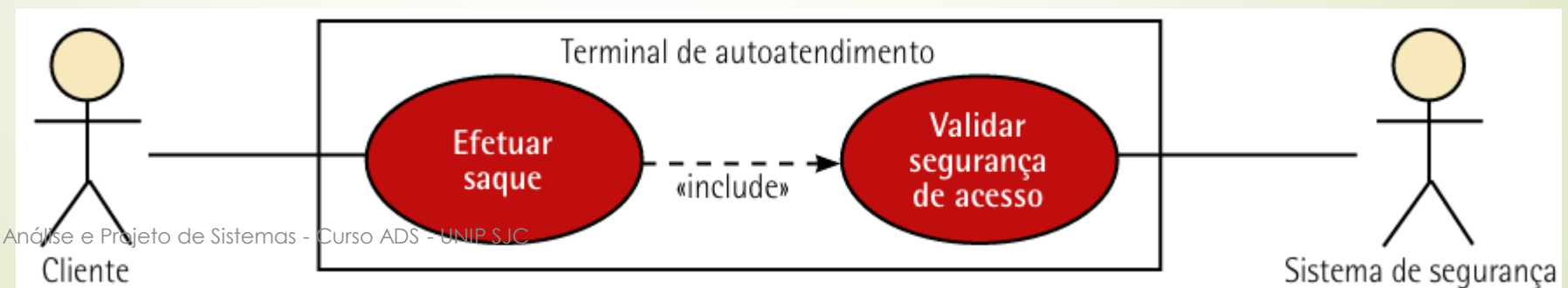
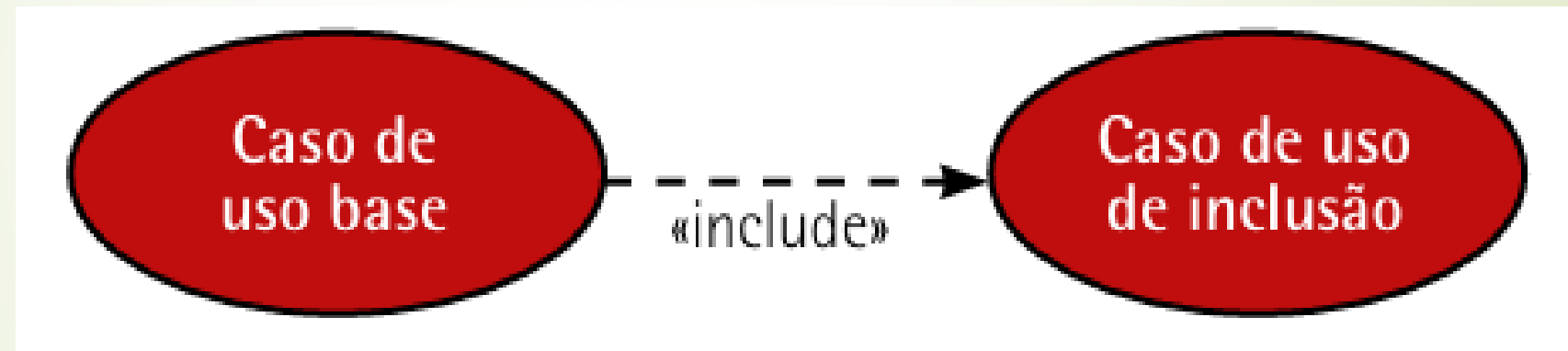
DCU - Relacionamentos

- relacionamento de inclusão entre casos de uso é mostrado como uma dependência (seta pontilhada) estereotipada com a palavra-chave include, conforme exibido a seguir:



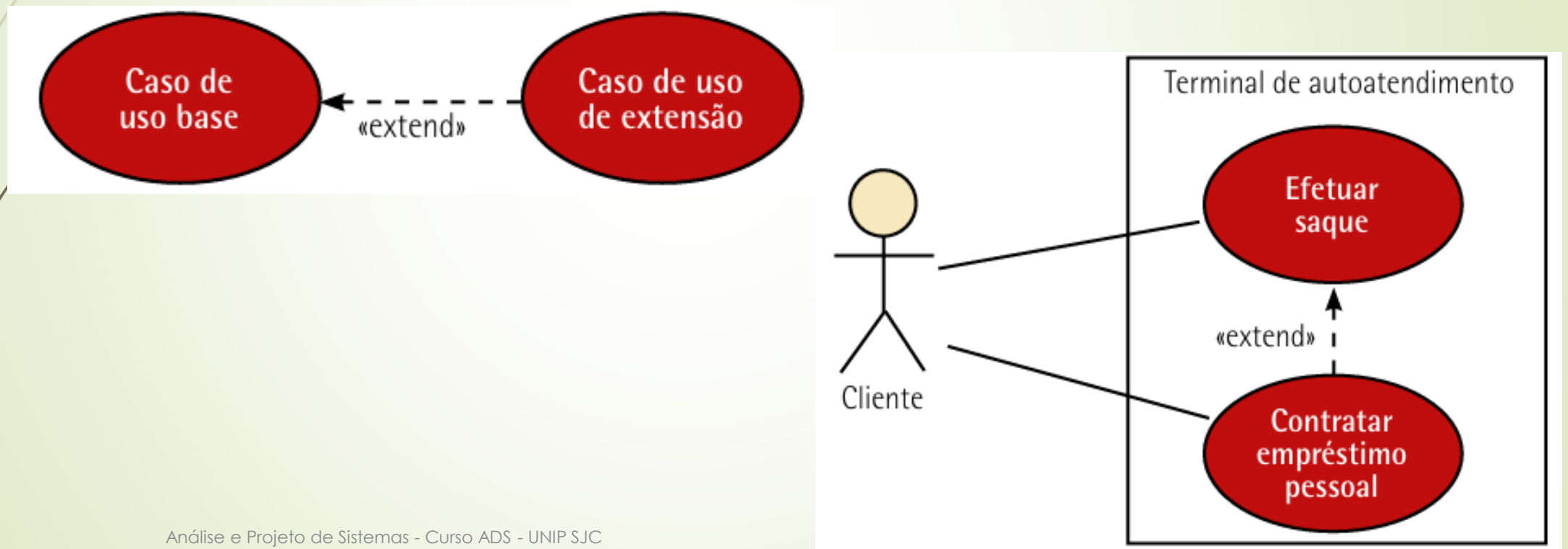
DCU - Relacionamentos

- relacionamento de inclusão entre casos de uso é mostrado como uma dependência (seta pontilhada) estereotipada com a palavra-chave include, conforme exibido a seguir:



DCU - Relacionamentos

- relacionamento de extensão entre casos de uso é mostrado como uma dependência (seta pontilhada) estereotipada com a palavra-chave extend, conforme exibido a seguir:



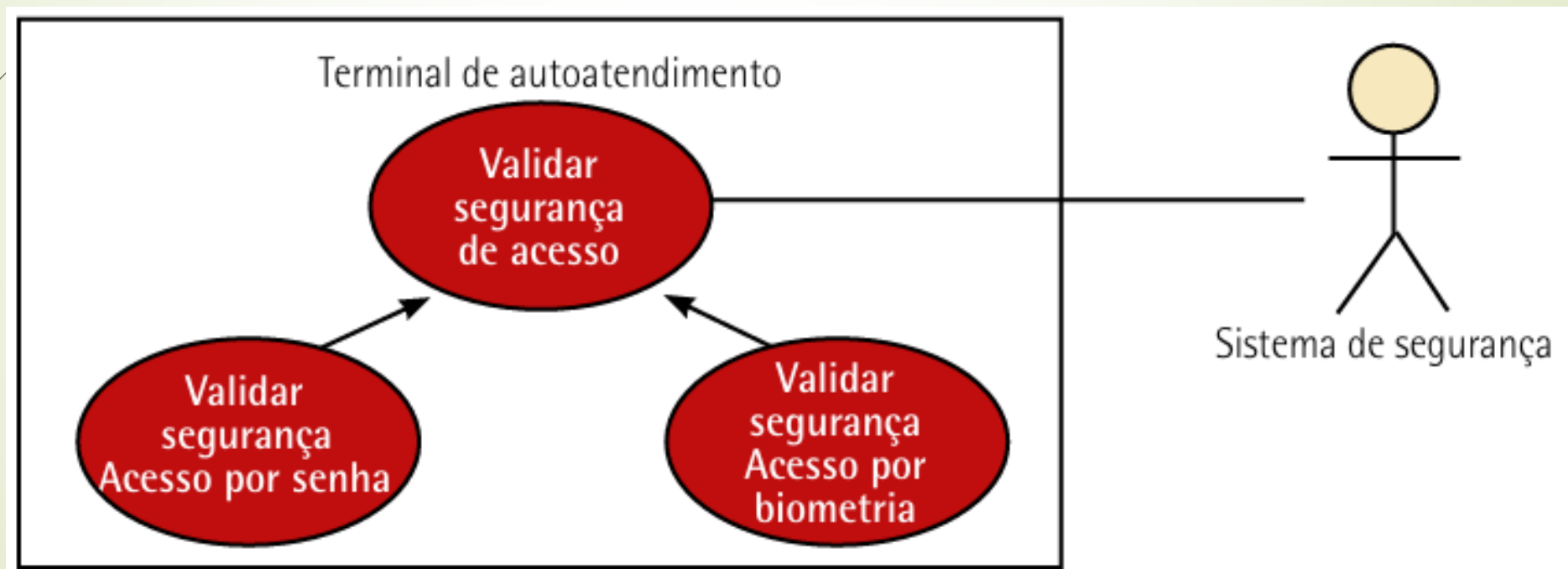
DCU – Relacionamentos - Herança

- Significa que o caso de uso filho herda o comportamento e o significado do caso de uso pai, acrescentando ou mudando seu comportamento
- Um relacionamento de herança entre casos de uso é mostrado com uma linha cheia com uma seta aberta, conforme exibido a seguir:



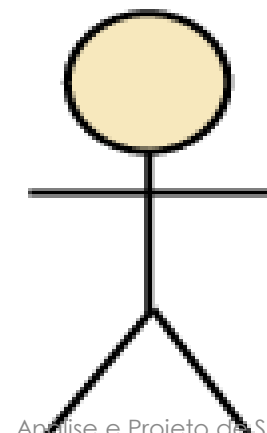
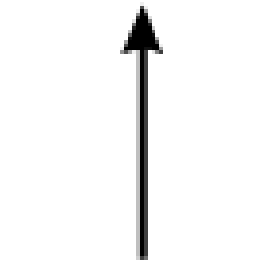
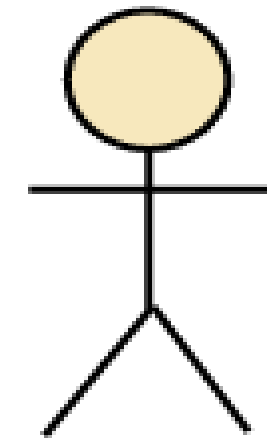
DCU – Relacionamentos - Herança

- podemos dizer que o caso de uso filho é uma especialização do caso de uso pai, ou que o caso de uso pai é uma generalização do caso de uso filho.



DCU – Relacionamentos - Herança

- Diferentemente dos casos de uso, atores possuem apenas um tipo de relacionamento: **herança**.
- Significa que o ator filho herda o comportamento e o significado do ator pai, acrescentando ou mudando seu comportamento.
- Em suma, o ator filho pode executar todos os casos de uso do ator pai e mais aqueles que apenas ele executa.

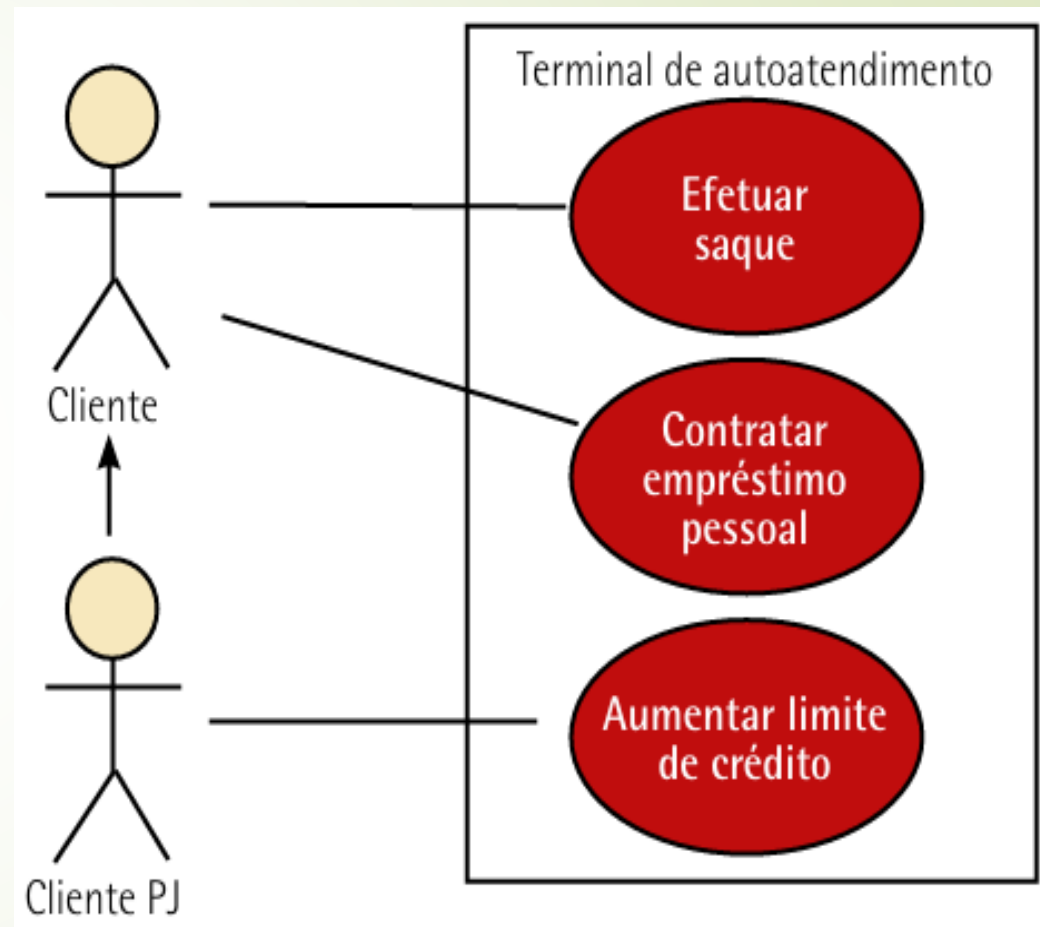


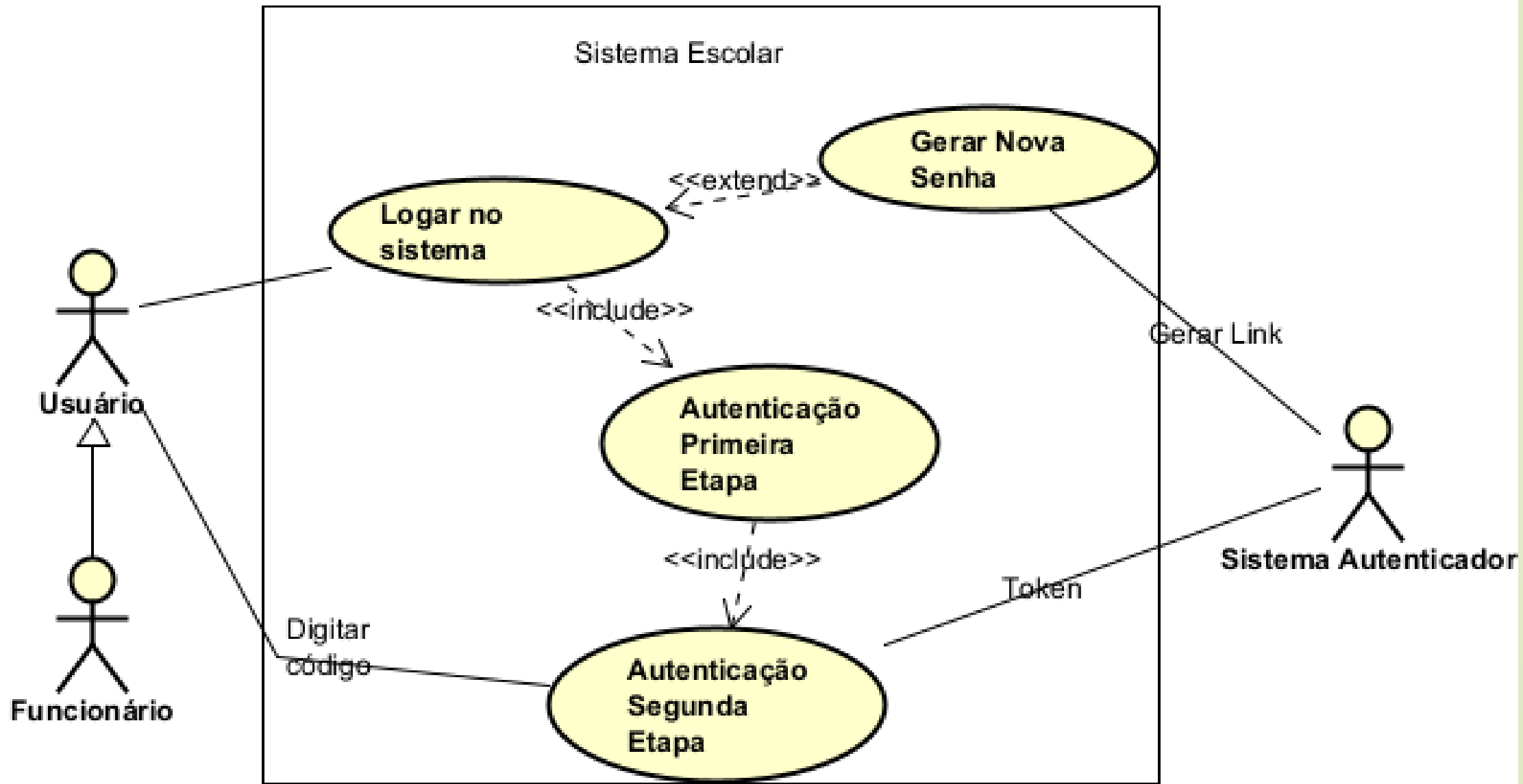
DCU – Relacionamentos Herança

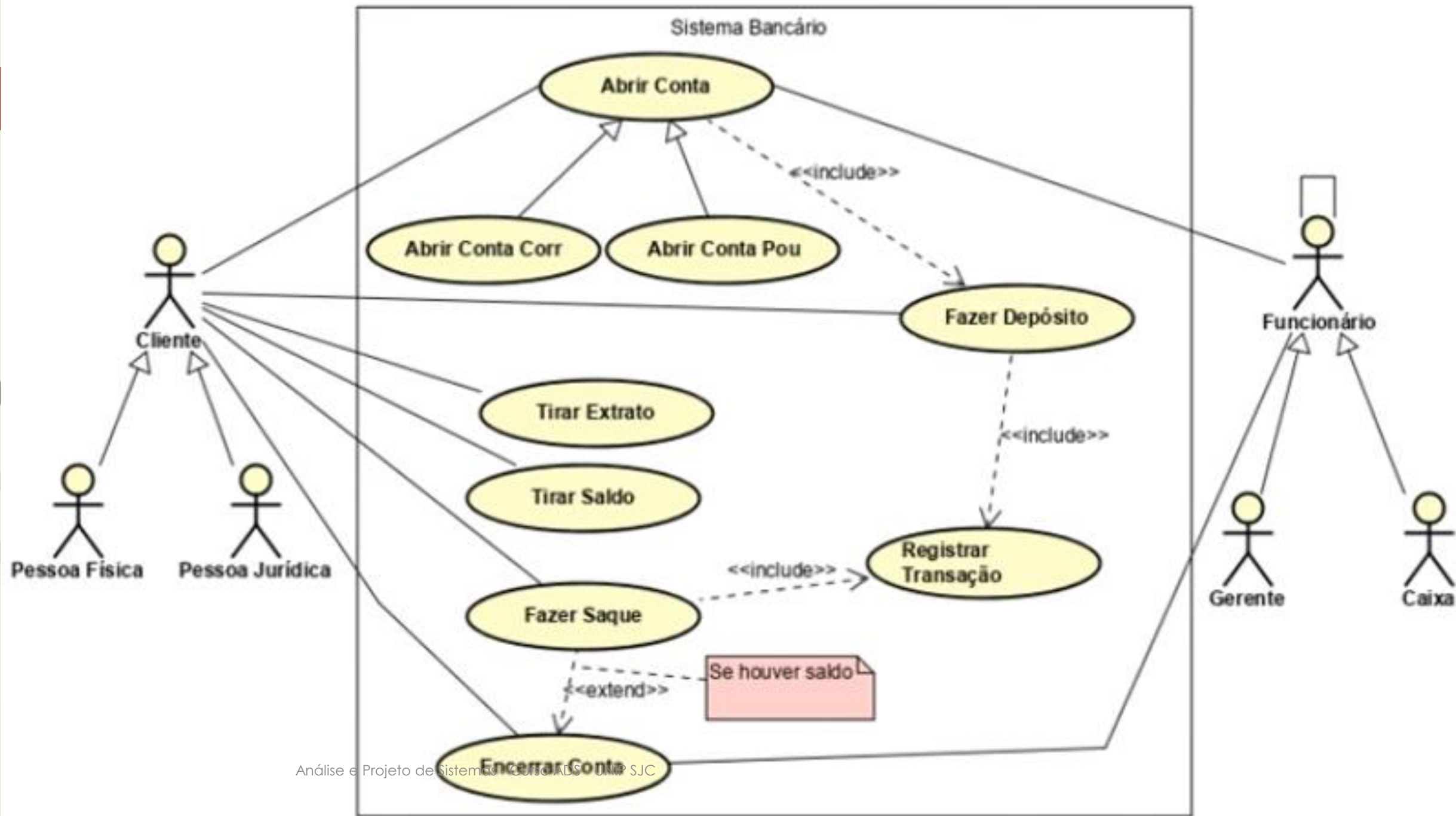
- Em UML, um relacionamento de herança entre atores é a mesma notação utilizada para cenários de uso: uma linha cheia com uma seta aberta, conforme exibido a esquerda:

DCU – Relacionamentos - Herança

- Por exemplo, podemos interpretar que o Cliente Pessoa Jurídica pode “Efetuar Saque”, “Contratar Empréstimo Pessoal” e “Aumentar Limite de Crédito”. Podemos afirmar que o ator Cliente não executa o caso de uso “Aumentar Limite de Crédito”, mas somente “Efetuar Saque” e “Contratar Empréstimo Pessoal”.









Fundamentos de Orientação a Objetos

APS – Análise e Projetos de Sistemas

Prof. Francisco Martins

OO – Visão geral

- Podemos dizer que o paradigma da orientação a objetos é uma forma de se desenvolver um sistema.
- Nesse paradigma, o software é visto como um conjunto de **componentes** que interagem entre si para resolver um determinado problema, **a cada componente dá-se o nome de objeto.**
- Às características de um objeto, damos o nome de **atributos**, e aos comportamentos, de **métodos.**

OO - Objeto

- Podemos afirmar que todo objeto possui uma identidade, representada por um conjunto de informações conferidas aos seus atributos, e que todo objeto possui um estado, definido também pelo conjunto dessas informações em um determinado espaço de tempo.
- Ex: Saldo da conta bancária.

OO - Classes

- Podemos definir classe, ou classe de objetos, como um grupo de objetos com mesmas propriedades (**atributos**), comportamento (**operações**), relacionamentos e semântica.
- Semanticamente, dizemos que uma classe é uma “especificação” de um objeto, pois nela está definido tudo o que o objeto possui (**atributos**) e tudo aquilo que o objeto pode fazer (**métodos**).
- Exs: Pessoa, Pedido, Cliente.

OO - Encapsulamento

- Encapsulamento significa deixar visível, ou deixar acessível a outros objetos, apenas o que é necessário.
- Por exemplo, podemos ocultar dos demais objetos de um sistema detalhes da implementação ou a lógica algorítmica de um método de um determinado objeto, uma vez que esses detalhes não são importantes para os demais, apenas para o objeto que possui essa lógica.
- Para os demais objetos que se comunicam com esse objeto, basta que ele faça o que deve fazer, não sendo importante o **como** será feito.

OO - Abstração

- Abstração é um dos principais conceitos aplicados à resolução de problemas, que é fundamental para a modelagem da estrutura de um sistema de software.
- Abstração está ligada à nossa capacidade de selecionar determinados aspectos do problema e **isolar o que é importante para algum propósito**, ou seja, **é dar ênfase àquilo que é necessário**.



OO – Diagrama de Classes

APS – Análise e Projetos de Sistemas

Prof. Francisco Martins

Modelo de Classe

Identificação da Classe

Lista de atributos

Lista de métodos

Cliente

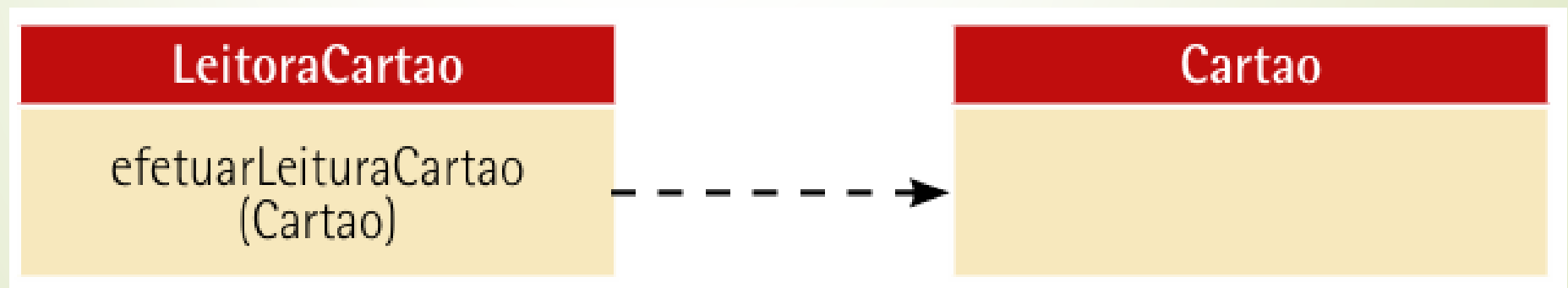
numeroContaCorrente: 1234
numeroAgencia : 0010
saldoContaCorrente : R\$ 500,00

inserirCartao()
informarSenha()
sacarContaCorrente()

O objetivo neste momento é reafirmar o conceito de que **um objeto é uma instância de uma classe**.

Relacionamento entre classes: Dependência

- Dependência é um relacionamento em que um objeto depende de informações de outro objeto para a execução de um determinado comportamento.
- Se dá quando uma classe utiliza informações de outra classe em um determinado método:



Relacionamento entre classes:

Associação

- A associação mostra que um objeto pode se relacionar com outro, que existe uma conexão entre esses objetos. Existem dois tipos de associação:
- • Associação binária: amplamente utilizada, é a associação entre duas classes apenas.
- • Associação enésima: raramente utilizada, é a associação entre mais de duas classes.

Relacionamento entre classes:

Associação

- Na UML, representamos a associação como uma reta, ou uma linha, que conecta as classes que se associam de alguma forma.



Relacionamento entre classes: Associação

Domínio	Representação UML			Interpretação
Vendas	Cliente	Compra	Produto	Um cliente compra produtos
Bancário	ContaCorrente	Possui	Transacao	Uma conta-corrente possui transações efetuadas
Hotelaria	Hospede	Ocupa	QuartoHotel	Um hóspede ocupa um quarto de hotel

- Além de identificar a relação entre as classes, estabelece o papel que cada objeto ou classe exerce nessa relação.

Opções de representação de multiplicidades de relacionamento

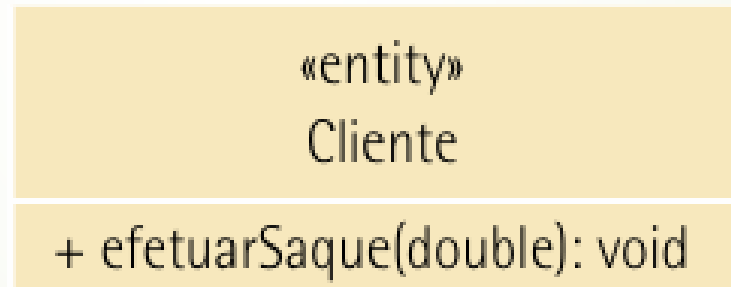
Representação	Interpretação
0..*	No mínimo zero e no máximo infinito OU Zero ou Muitos
1..*	No mínimo um e no máximo infinito OU Um ou Muitos
0..1	No mínimo zero e no máximo 1 OU Zero ou Um
1	Apenas Um
X..Y	Intervalo específico de no mínimo X e no máximo Y

Visibilidade dos Membros (Atributos / Métodos)

- Visibilidade indica quando e em que nível um atributo ou um método de um objeto pode ser acessível aos objetos que se relacionam com ele.
- Em orientação a objetos, temos os níveis de visibilidade de atributos e métodos como segue:
 - **+ Público**
 - **- Privado**
 - **# Protegido**
 - **~ Pacote**
 - **/ Derivado**
 - *** Em destaque os mais usados**

Visibilidade dos Membros (Atributos / Métodos)

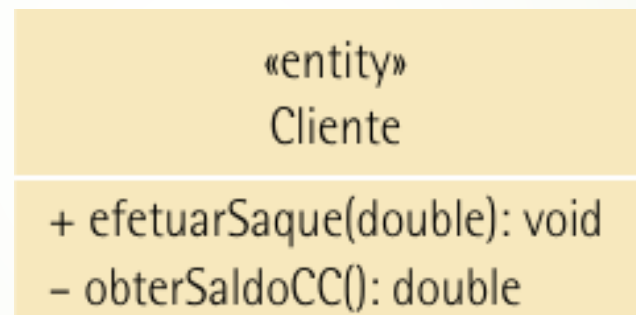
- Público: o atributo ou o método pode ser acessado por qualquer classe. Na UML, indicamos que um atributo ou um método é público utilizando o sinal + (positivo), conforme exemplo a seguir.



- Podemos dizer que o método efetuarSaque() pode ser chamado por qualquer outro objeto.

Visibilidade dos Membros (Atributos / Métodos)

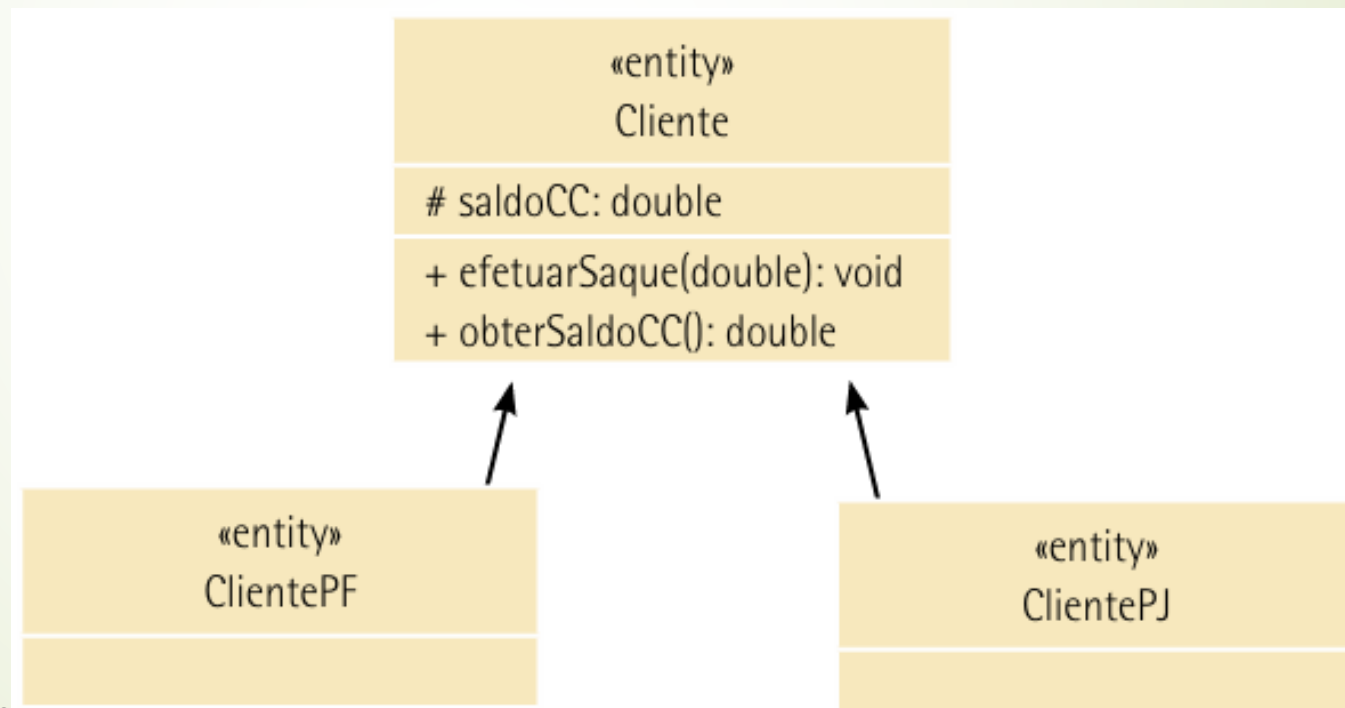
- Privado: um atributo ou método privado pode ser acessado somente na própria classe em que está declarado. Na UML, indicamos que um atributo ou um método é privado utilizando o sinal – (negativo), conforme exemplo a seguir.



- Podemos dizer que o método obterSaldoCC() pode ser chamado apenas dentro da própria classe Cliente. Por exemplo: poderíamos ter uma chamada a esse método dentro do método efetuarSaque().

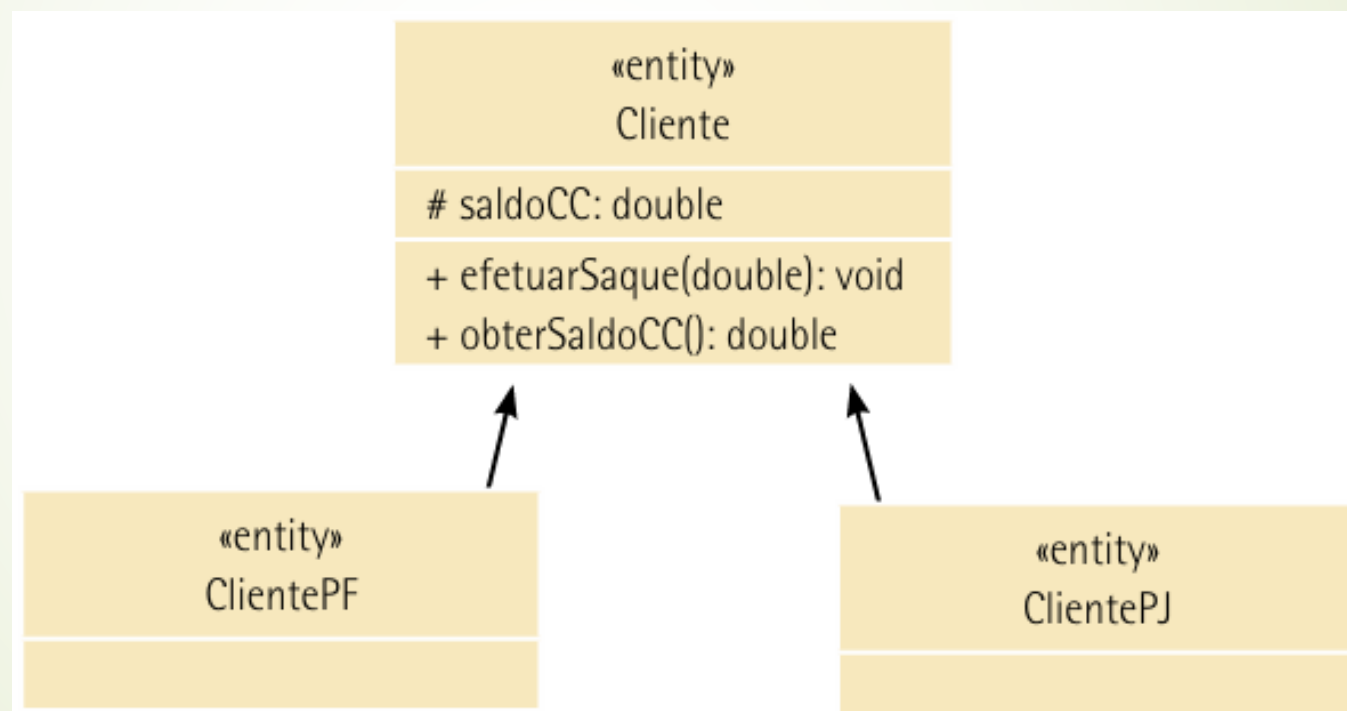
Visibilidade dos Membros (Atributos / Métodos)

- Protegido: um atributo ou um método protegido pode ser acessado apenas na classe em que está declarado e em suas classes filhas. Na UML, indicamos que um atributo ou um método é protegido utilizando o sinal # (sustenido), conforme exemplo a seguir.



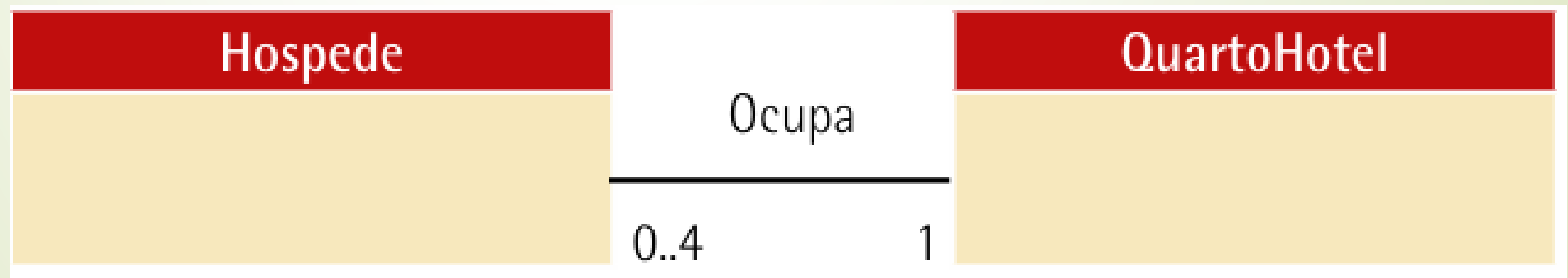
Visibilidade dos Membros (Atributos / Métodos)

- Podemos dizer, de acordo com a figura, que o atributo saldoCC é acessível apenas na classe Cliente e nas suas classes filhas: ClientePF e ClientePJ.



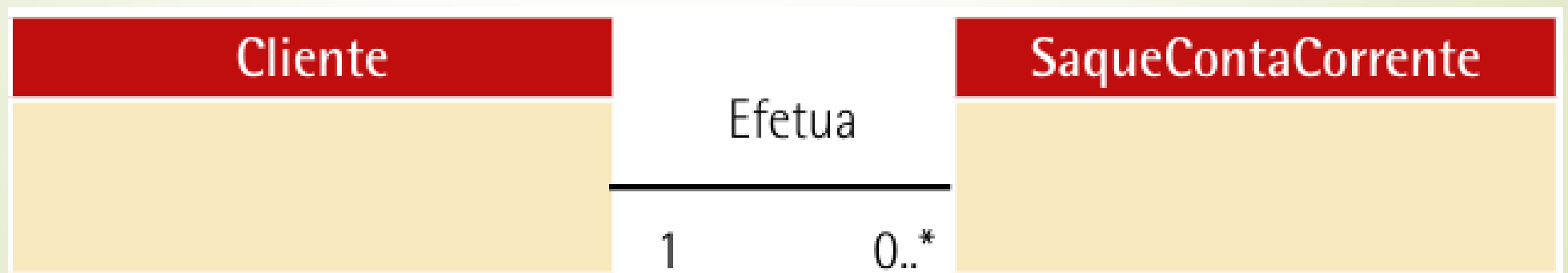
Associação: Hospede e Quarto de Hotel

- Nesta figura, temos duas interpretações possíveis a partir da representação, ambas corretas:
- • Um hóspede ocupa no máximo um quarto de hotel.
- • Um quarto de hotel pode ser ocupado por no mínimo zero ou no máximo quatro hóspedes.



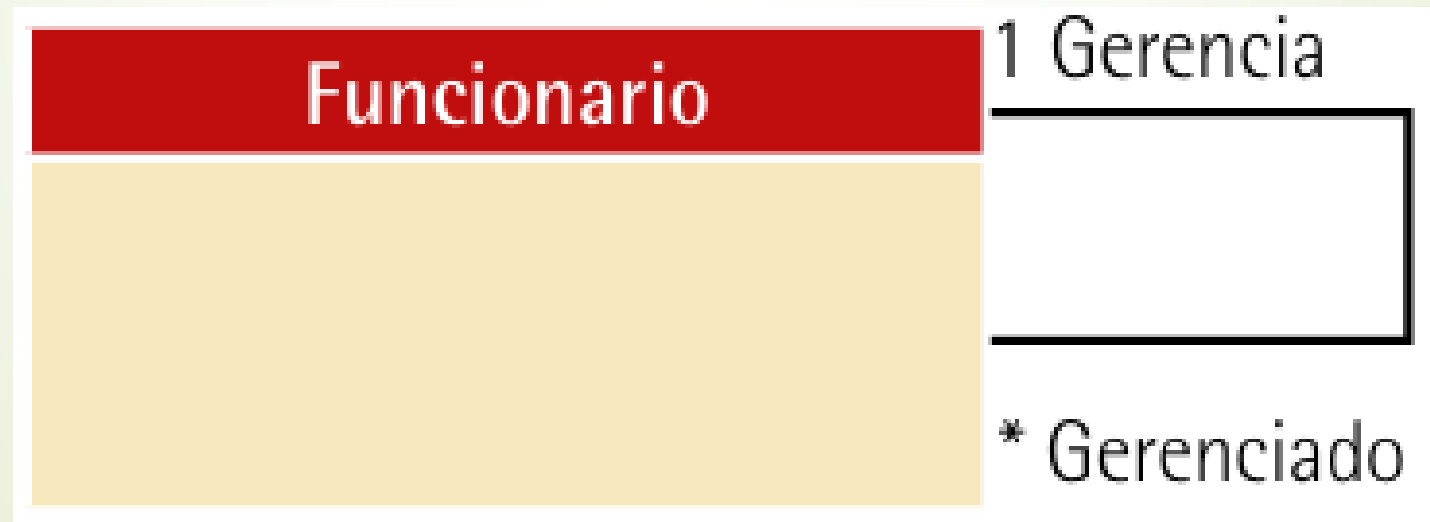
Associação: Cliente do banco

- Temos duas interpretações possíveis a partir da representação:
- • Um cliente pode efetuar no mínimo zero ou no máximo infinitos saques.
- • Uma operação de saque de conta-corrente é feita por apenas um cliente.



Associação: Funcionário

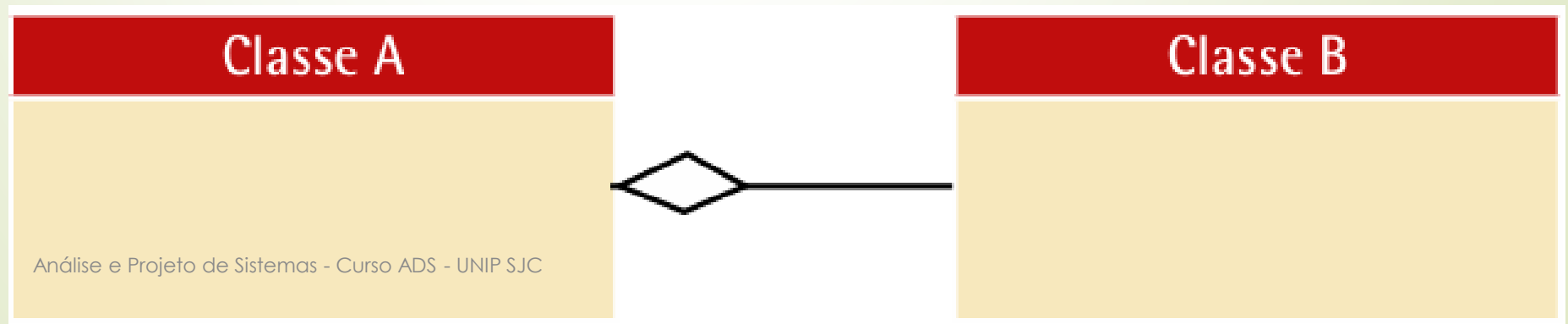
- Existe ainda outra variação de associação, chamada associação reflexiva, que é quando objetos da mesma classe são associados, sendo que nessa associação esses objetos possuem papéis distintos



Agregação

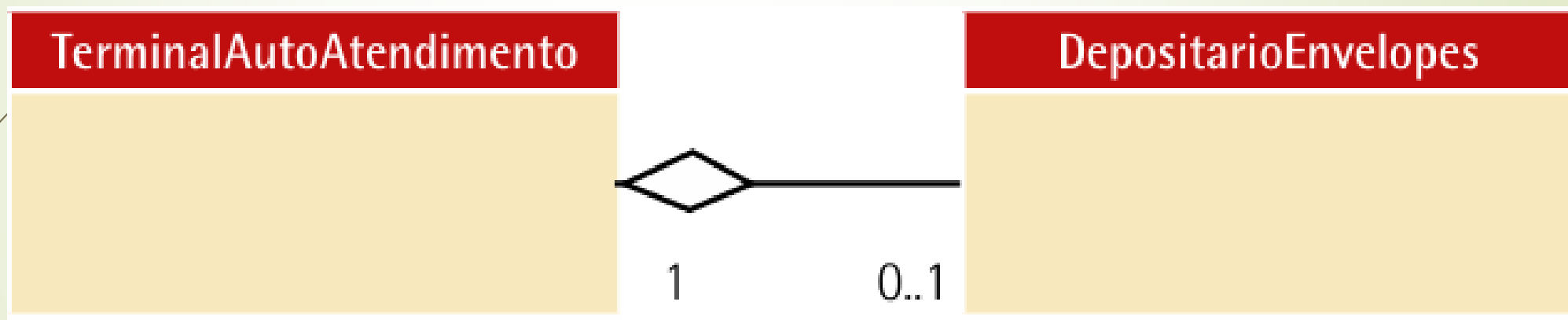
43

- Na UML, representamos agregação utilizando uma reta saindo da classe que representa a parte e se conectando a um losango, também chamado de diamante, na classe que representa o todo.
- Conceitos fundamentais da agregação podem ser interpretados:
 - • A Classe B é parte da Classe A e o inverso nunca é verdadeiro.
 - • A existência de um objeto da Classe B é independente da existência de um objeto da Classe A.



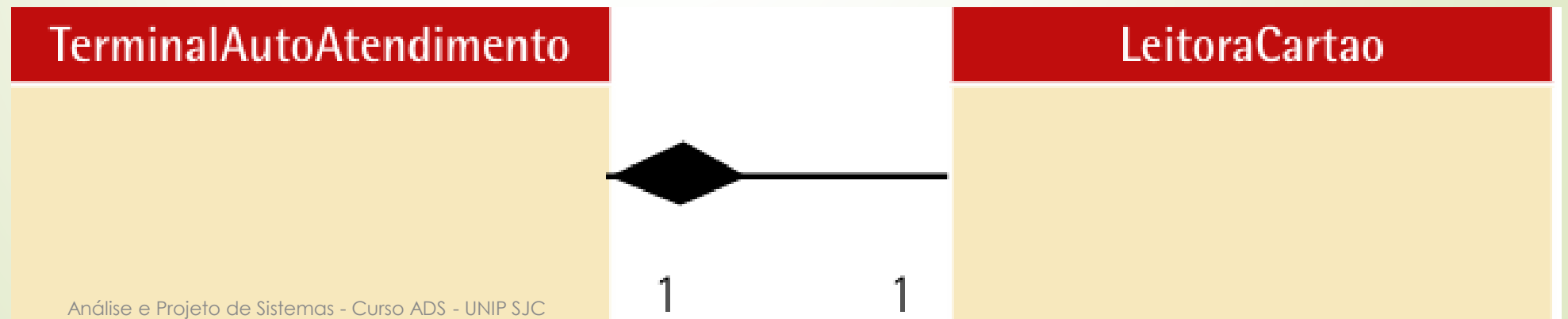
Agregação

- Assim como na associação, na agregação temos o conceito de multiplicidade, que define a quantidade de objetos agregados na relação todo-parte.



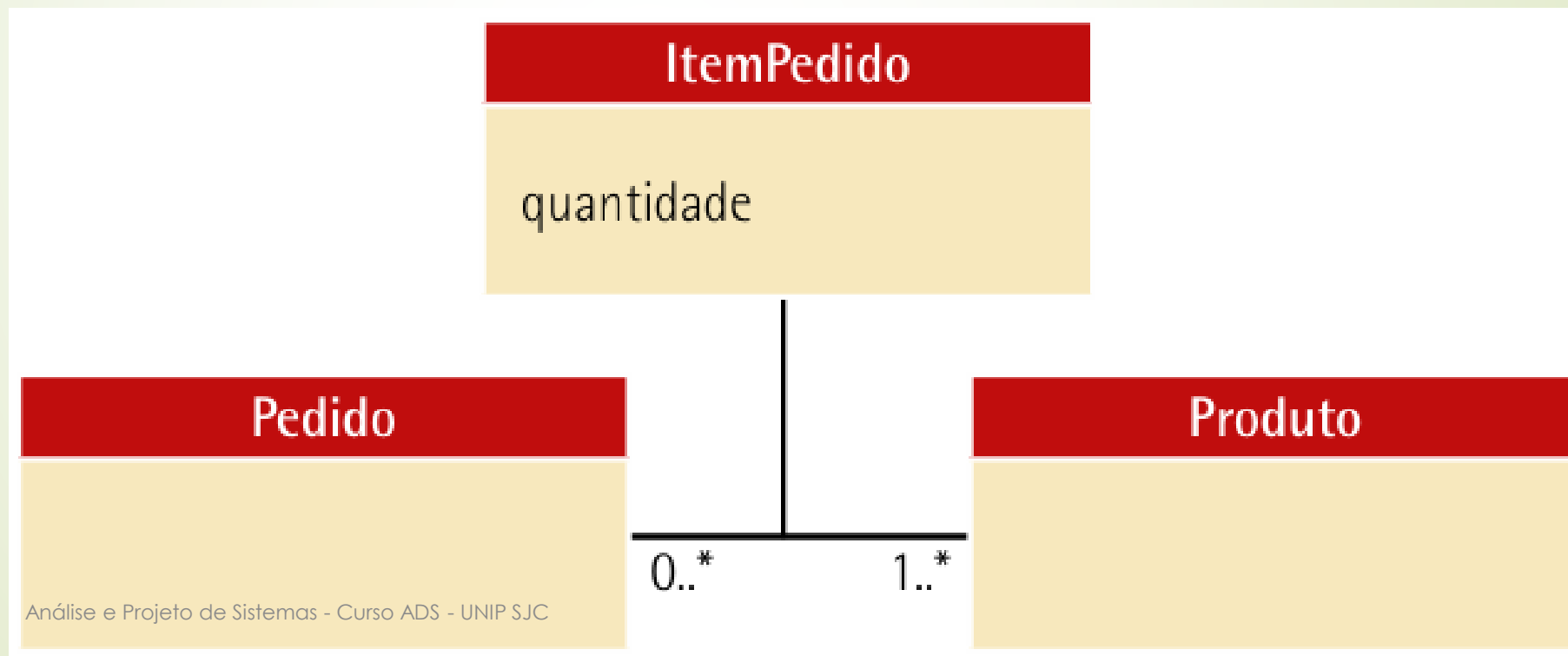
Composição

- A composição tem exatamente o mesmo conceito da agregação, ou seja, estabelece uma relação todo-parte entre dois objetos.
- A única diferença entre composição e agregação é que a classe que representa a **parte** da relação, para existir, depende da classe que representa o **todo**, ou seja, o ciclo de vida do objeto da classe parte depende do ciclo de vida do objeto da classe todo.



Classe Associativa (N:M ou Muitos p/ Muitos)

- Na UML, representamos uma classe associativa da mesma forma que representamos uma classe “normal”, contudo, ela é ligada por uma linha tracejada ou não, que termina na linha que define a associação de duas outras classes.

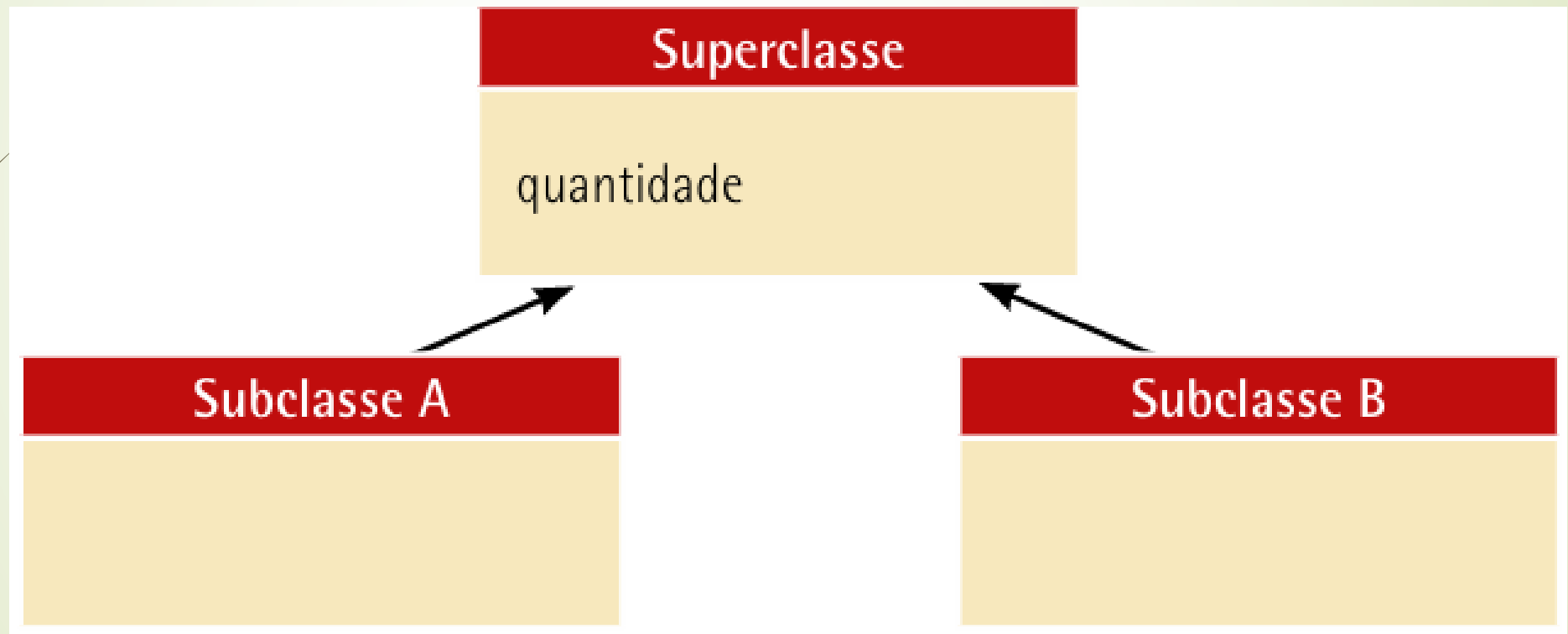


Herança

- Herança é um dos conceitos mais importantes e mais utilizados no paradigma da orientação a objetos.
- Podemos dizer ainda que, se um objeto é uma instância de uma classe ou uma classe é uma especificação de um objeto, uma classe herda atributos e métodos de outra. Nessa situação temos dois papéis de classes na relação: classe mãe e classe filha.
- Classe mãe, ou superclasse, possui atributos e métodos que podem ser herdados por uma ou mais classes filhas, também chamadas de subclasses.

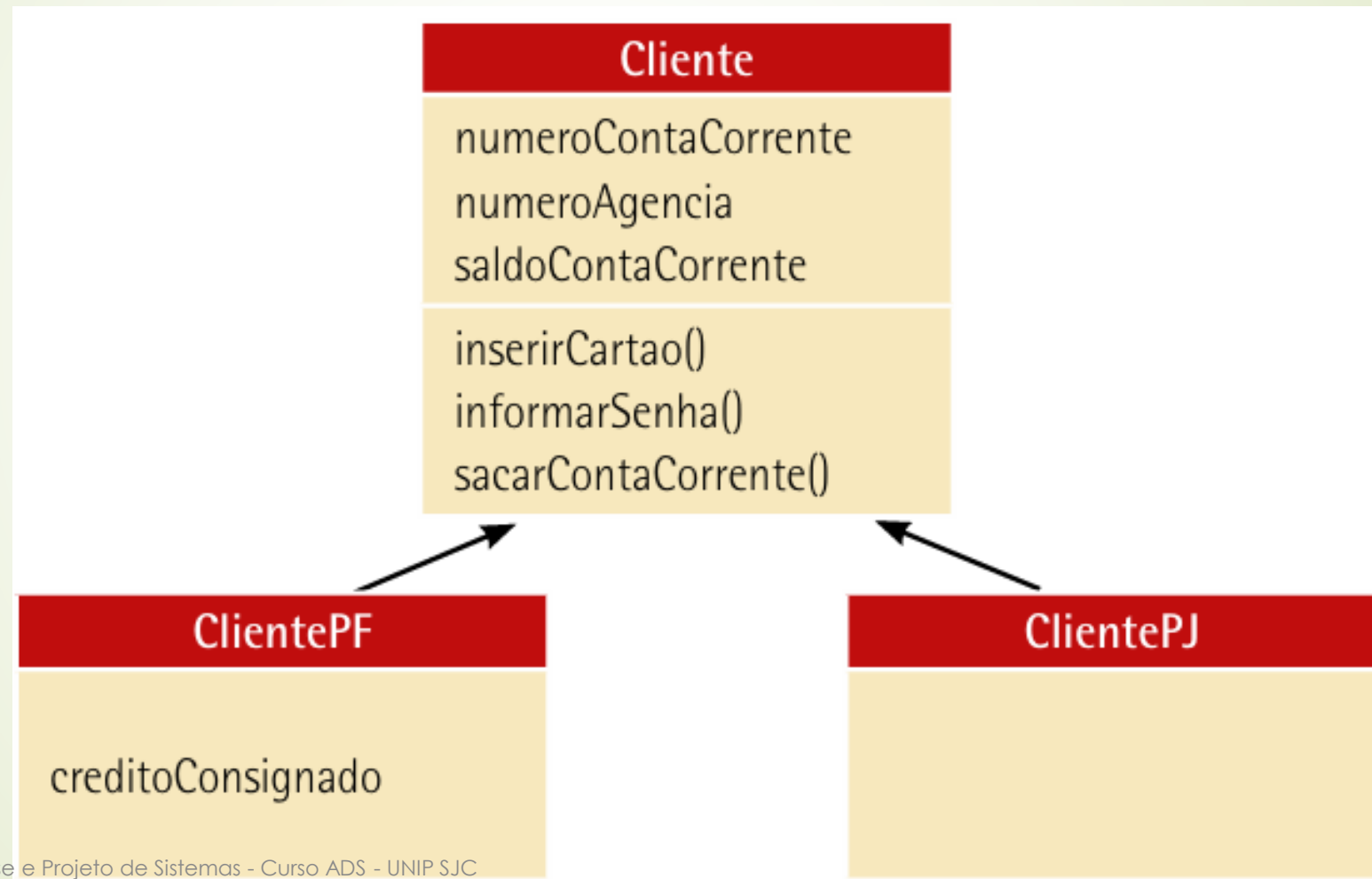
Herança

- Na UML, representamos herança com uma seta direcional, sendo que a ponta da seta é cheia, como mostra a figura.



Herança

- Exemplo de representação de herança Na UML.



Herança múltipla

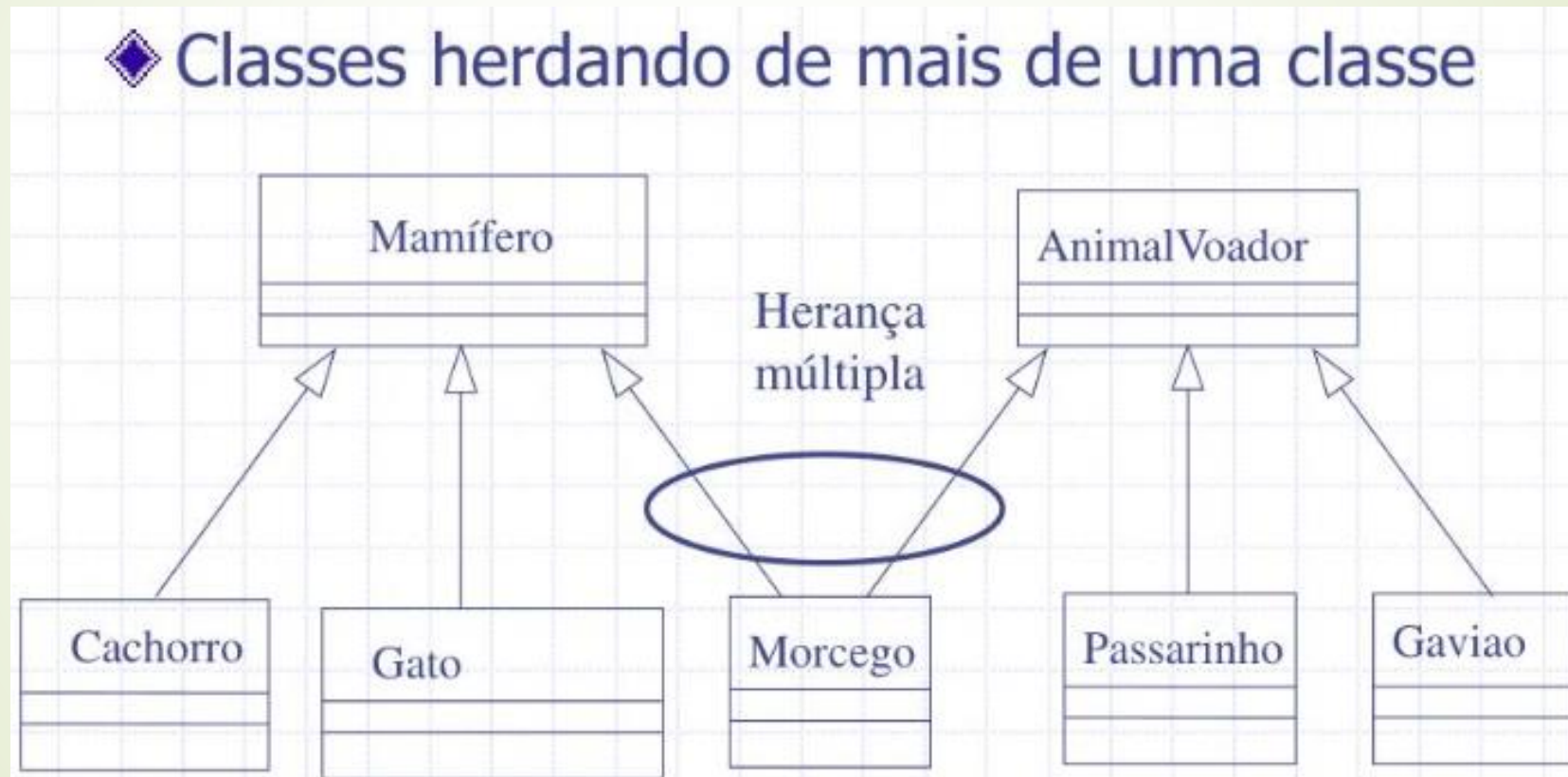
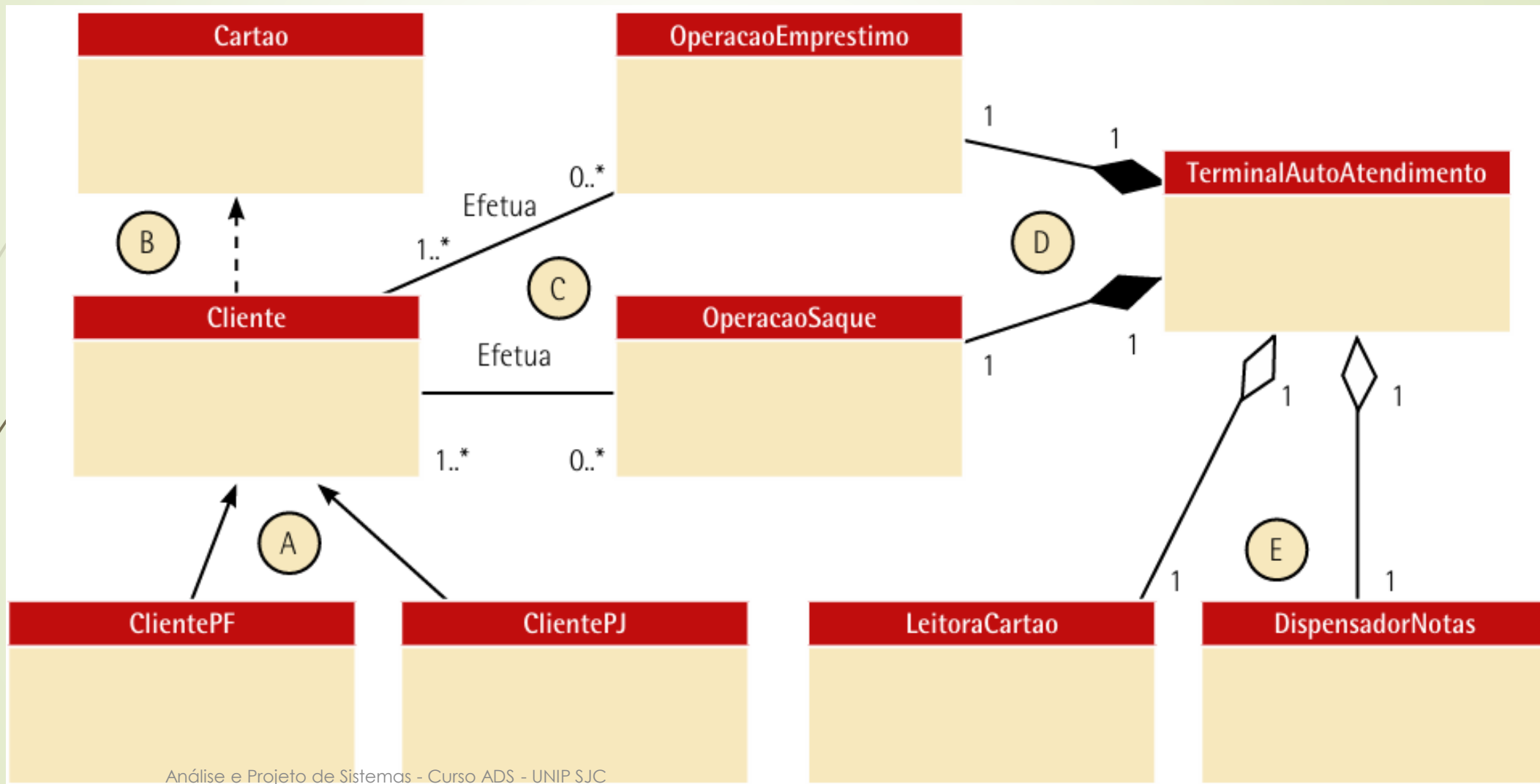


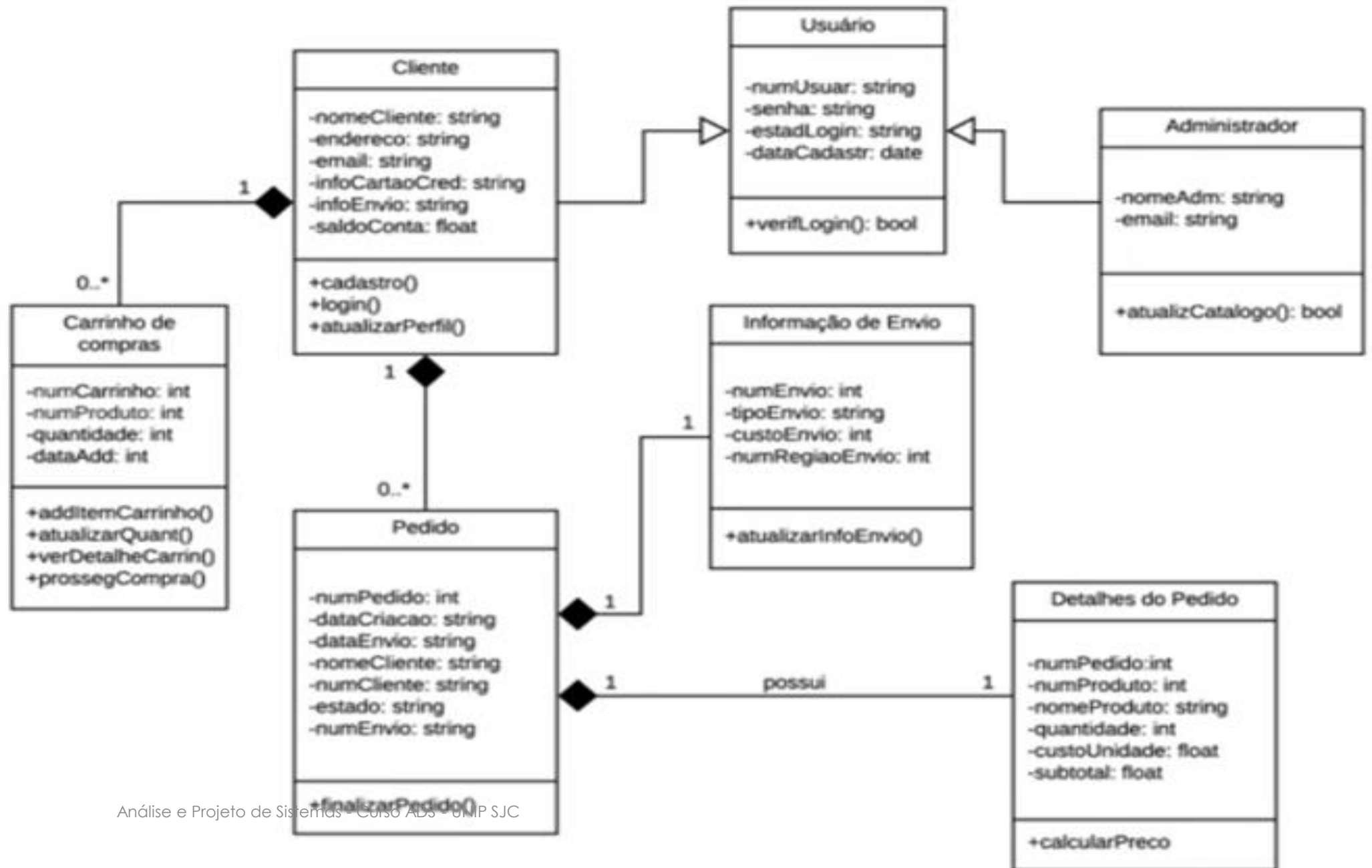
Diagrama de classes do saque em autoatendimento

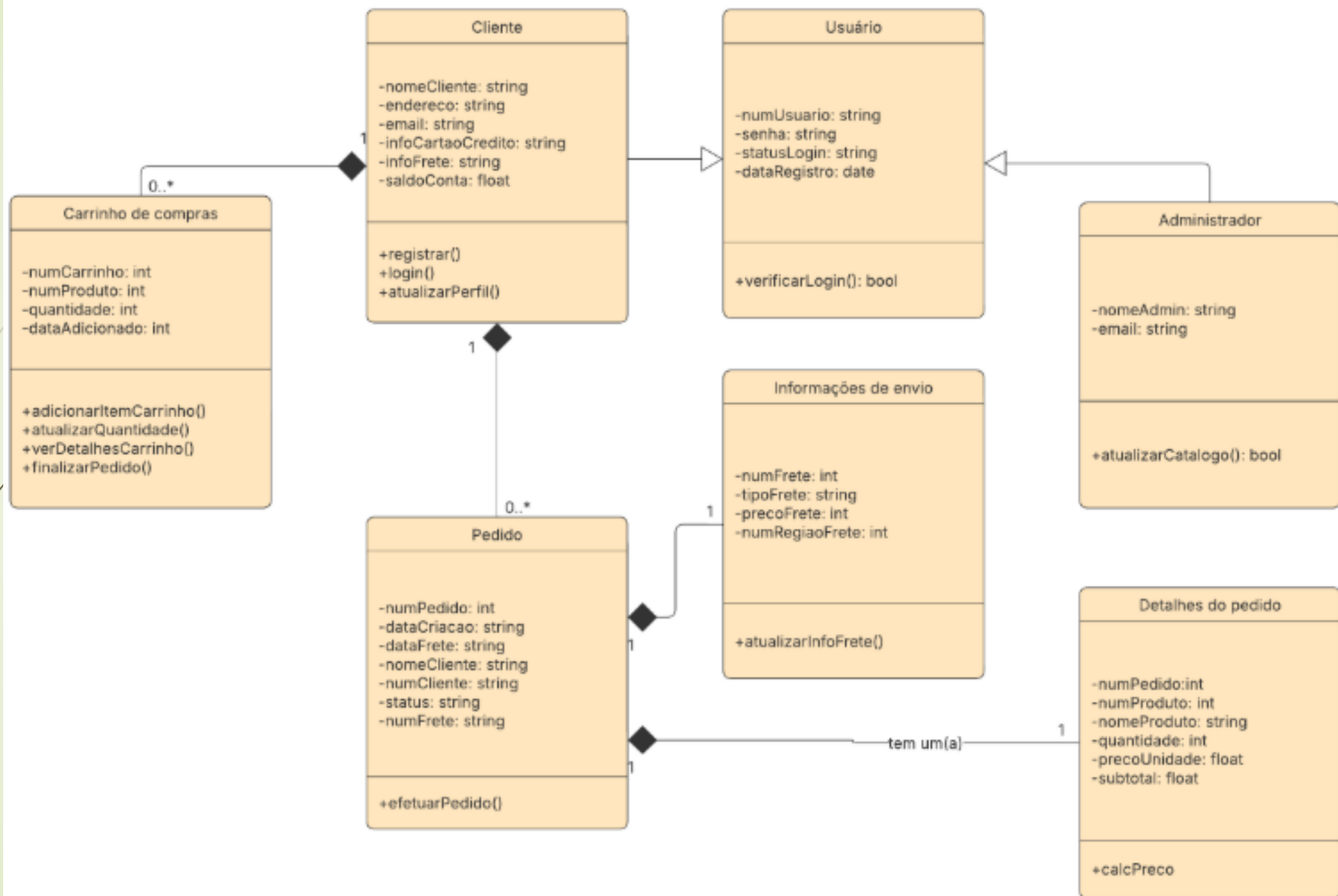
51



Carrinho de compras online

52







OO – Diagrama de Sequência

APS – Análise e Projetos de Sistemas

Prof. Francisco Martins

Diagrama de sequência

- Enfatiza a ordenação temporal das mensagens. No geral, se baseia em um caso de uso definido pelo diagrama de mesmo nome, e usa o diagrama de classes para definir os objetos envolvidos no processo
- As interações são mostradas na ordem em que elas ocorrem, desta forma esse tipo de diagrama mostra uma sequência de eventos.
- Os Caso de Uso são refinados em um ou mais Diagramas de Sequência

Aplicações dos Diagramas de Sequência

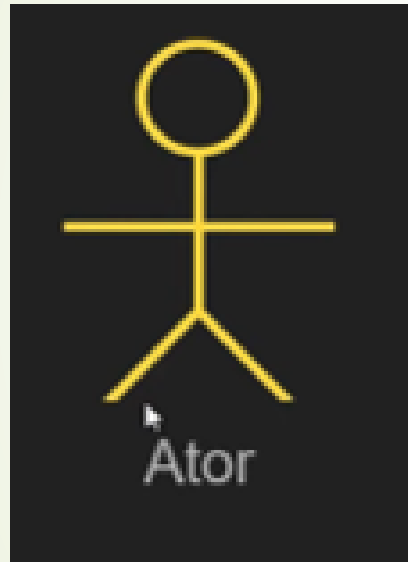
- Ajudar a identificar como os objetos no sistema interagem uns com os outros para implementar as funcionalidades definidas.
- Visualizar como mensagens são trocadas entre os componentes.
- Entender os requisitos do sistema ou parte dele.
- Realizar a documentação de processos em um sistema em desenvolvimento.
- Mostrar detalhes em diagrama de Casos de Uso.

Diagrama de sequência:

Componentes básicos

- ▶ Atores
- ▶ Objetos que participam da interação
- ▶ Mensagens trocadas (e seus parâmetros)
- ▶ Linha de vida e Caixa de Ativação
- ▶ Operadores de Controles Estruturado
- ▶ Foco de controle
- ▶ Gate

Diagrama de sequência: Atores



- Representam papéis que interagem com o sistema e seus objetos.
- Um ator está sempre fora do sistema modelado.
- Os atores são empregados para representar usuários humanos e outros elementos externos ao sistema.

Diagrama de sequência: Objetos

- São elementos que participam das interações no diagrama. Representam uma classe ou um objeto
 - Interação por meio de mensagens.
 - Colocados no nível superior do diagrama, ao longo do eixo x.
 - Objeto que inicia a operação é colocado à esquerda.
 - Objetos subordinados à direita
- Um objeto é representado por um retângulo com seu nome.

Dispositivo

Diagrama de sequência: Linha de Vida

- Elemento nomeado que representa um participante interno individual na interação.
- A Linha de Vida de um objeto é um linha tracejada vertical que representa o período de tempo no qual um objeto existe.

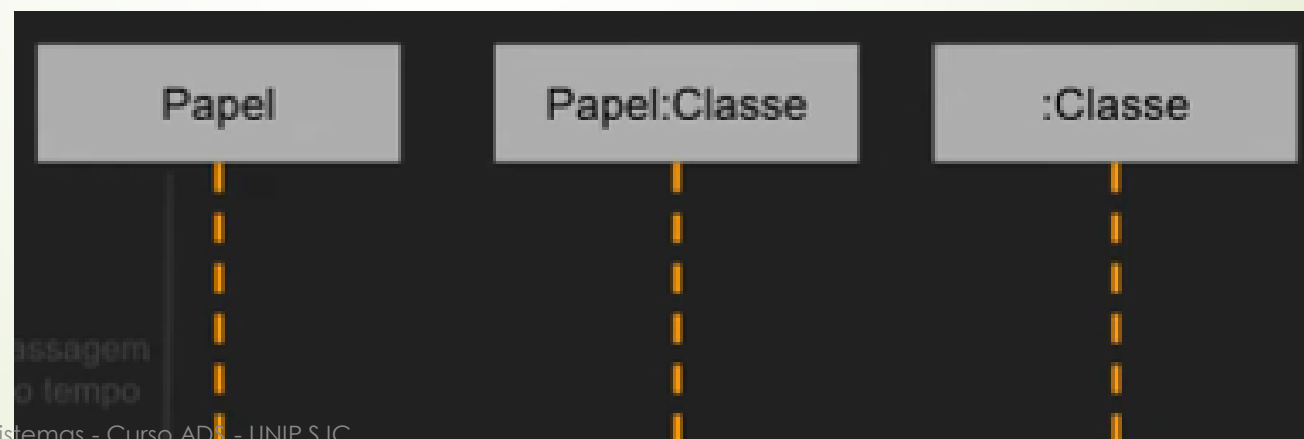


Diagrama de sequência: Foco de Controle

- ▶ Período no qual um objeto está participando ativamente de um processo, enquanto não termina uma tarefa. Também chamado de **Caixa de Ativação**.
- ▶ Representado dentro da linha de vida de um objeto, como um retângulo na vertical.



Diagrama de sequência: Mensagens

- Demonstram a ocorrência de eventos (chamadas de métodos) ou comunicação entre objetos, sem chamar métodos.
- Mostram as informações que são enviadas entre objetos.
- As mensagens são representadas por uma seta horizontal que vai de uma linha de vida a outra, apontando para o destinatário da mensagem

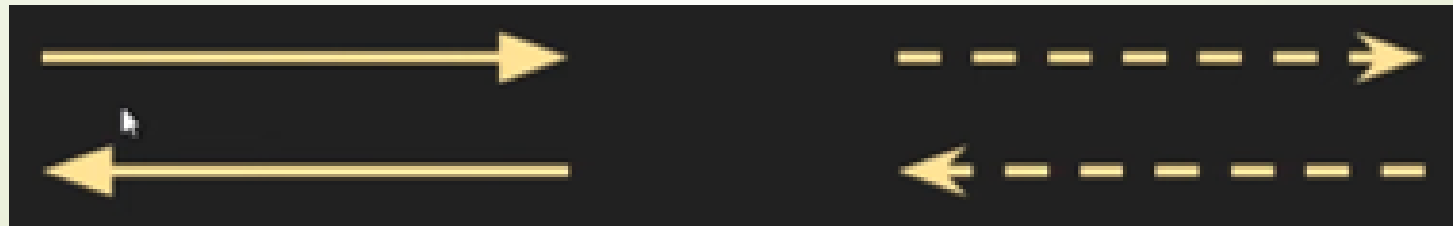
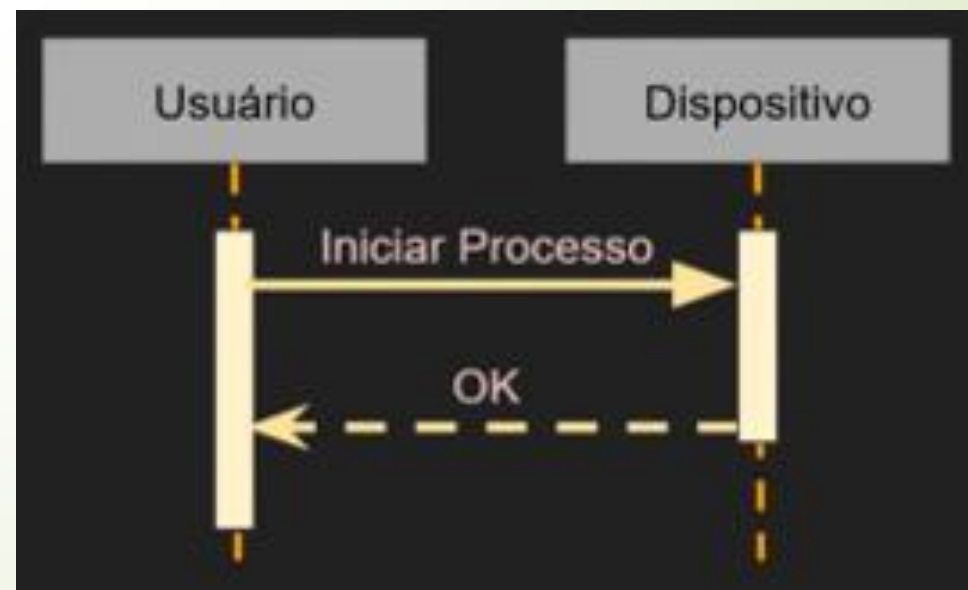


Diagrama de sequência: Tipos Mensagens

- Existem vários tipos de mensagens empregadas em diagramas de sequência, sendo as mais comuns as seguintes:
 - **Mensagem Síncrona**
 - **Mensagem Assíncrona**
 - **Auto-mensagem**
 - **Mensagem de Resposta**
 - **Mensagem de criação e de exclusão de Participante**
 - **Mensagem de Guarda**

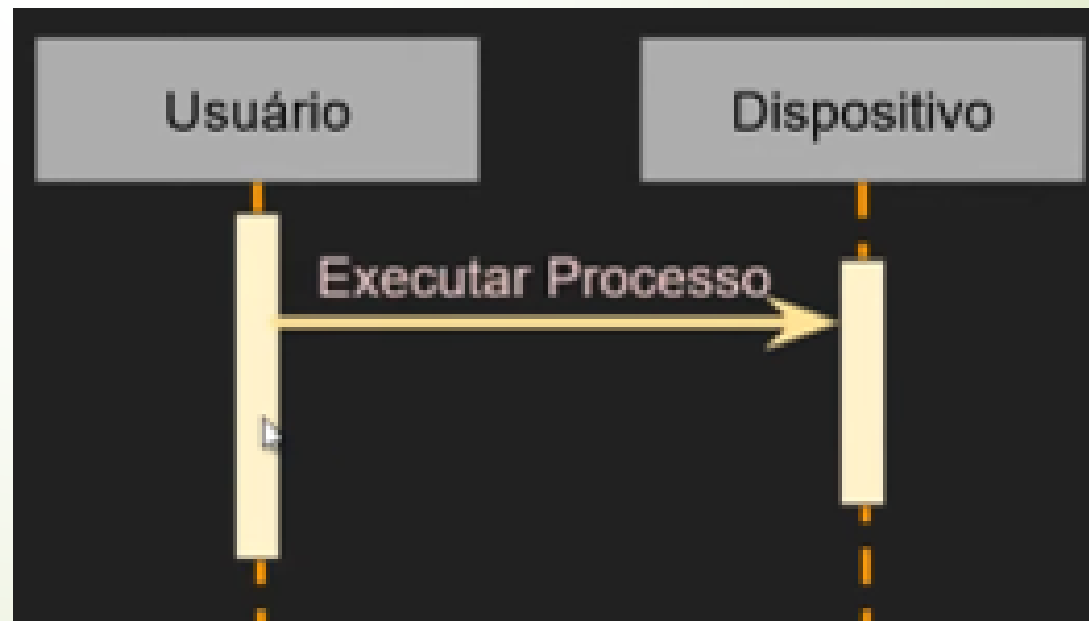
Mensagem Síncrona

- Mensagem que espera por uma resposta antes que a interação possa prosseguir. O remetente espera até que o processador tenha terminado o processo. Grande parte das mensagens é síncrona.
- Usamos uma seta de ponta sólida para representar a mensagem síncrona.



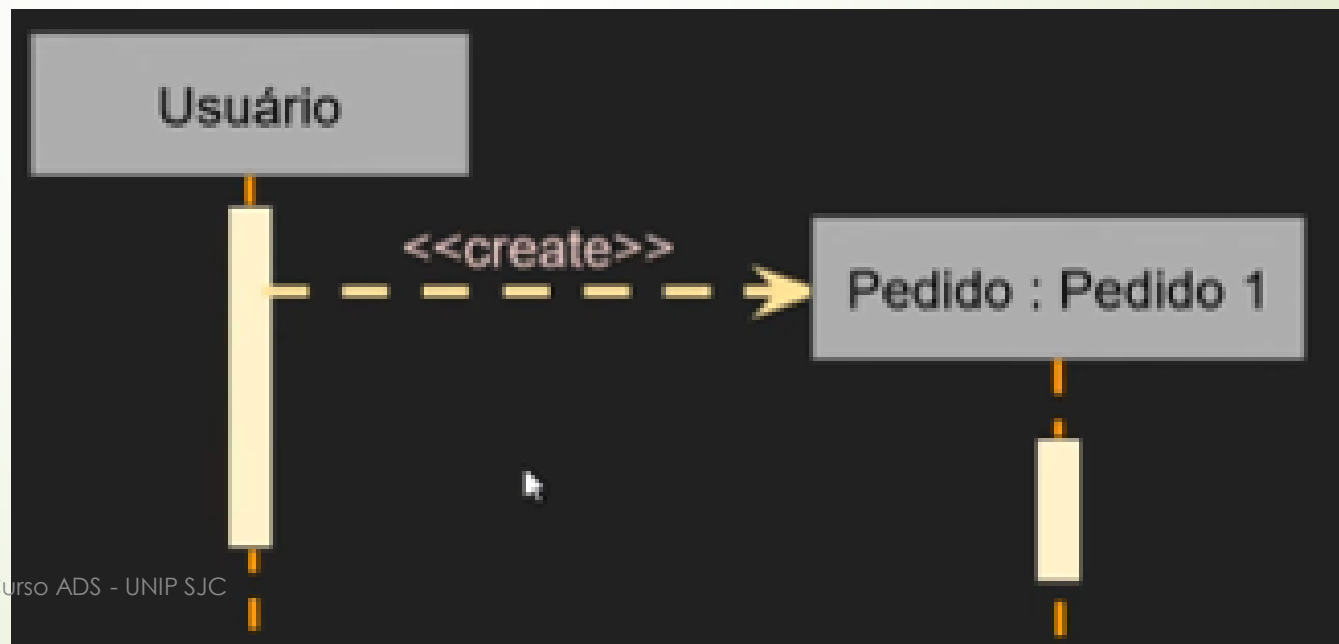
Mensagem Assíncrona

- Mensagem que não espera por uma resposta do destinatário antes que a interação possa prosseguir. A interação prossegue independente do destinatário processar a mensagem ou não.
- Usamos uma seta de ponta linear para representar a mensagem assíncrona.



Mensagem de Criação de Participante

- Uma mensagem de criação permite instanciar um novo objeto no diagrama. Às vezes, é necessário que uma mensagem crie um novo objeto.
- Representada por uma seta tracejada com o rótulo
- **<<create>>** em uma tag.



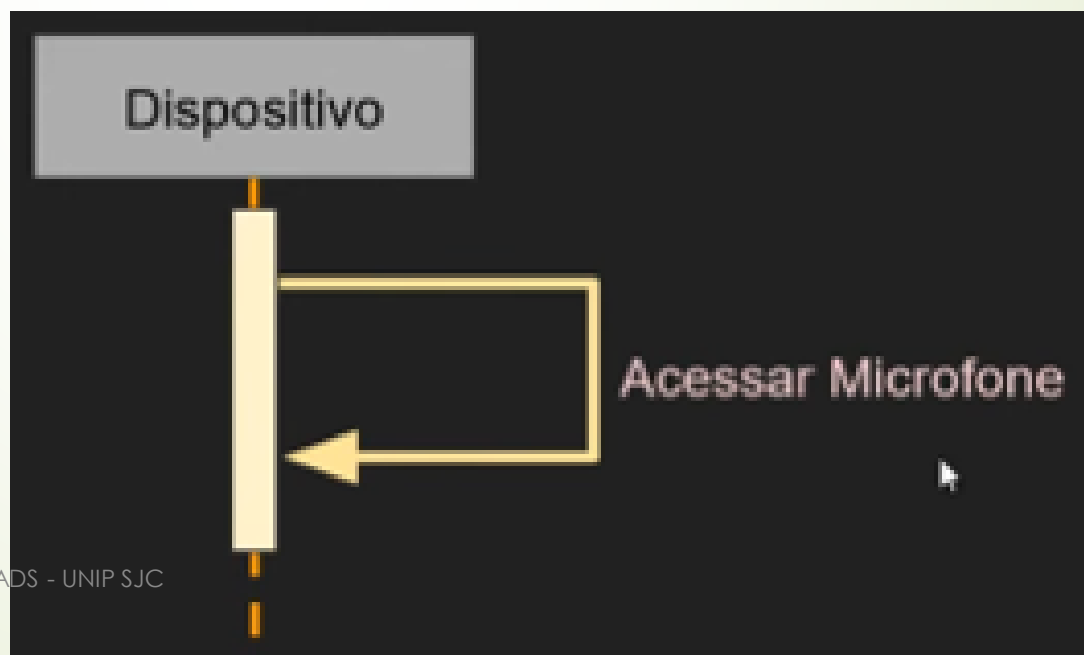
Mensagem de Exclusão de Participante

- A mensagem de exclusão é usada para eliminar um objeto. Assim, a memória é desalocada pela destruição da ocorrência do objeto no sistema.
- Representada por uma seta que termina com um X.



Auto-Mensagem

- É usada quando um objeto necessita realizar uma chamada a si mesmo.
- Representada por uma seta que sai do objeto e volta para o mesmo objeto.
- Pode ser síncrona ou assíncrona.



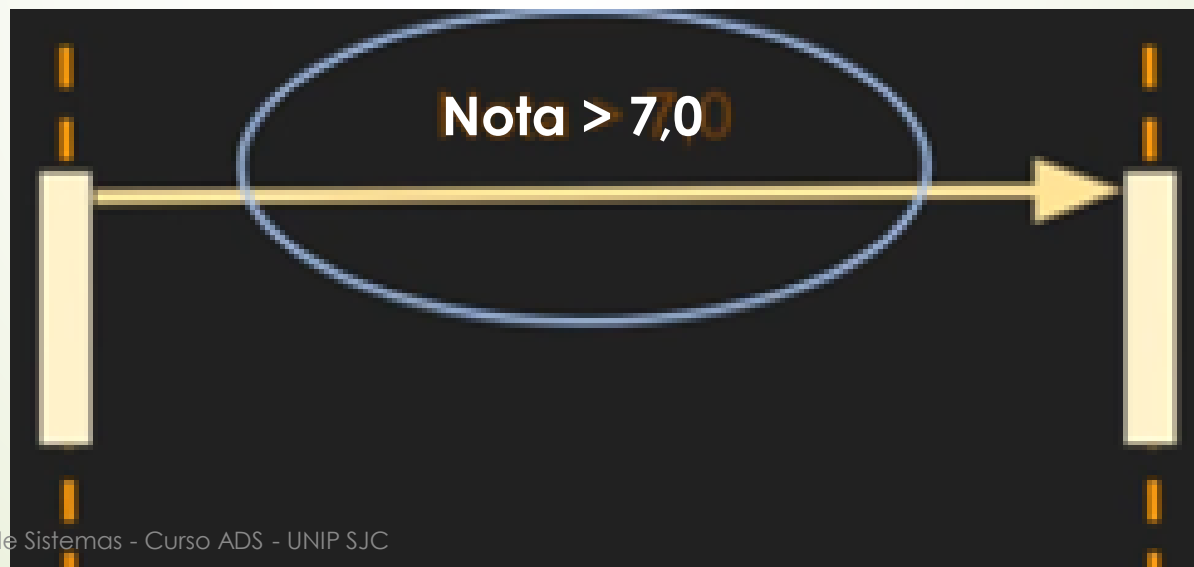
Mensagem de Resposta (ou de retorno)

- Identifica a resposta a uma mensagem enviada para um objeto.
- Pode retornar informações específicas ou apenas êxito ou falha na execução de um método.
- Representada por um alinha tracejada que aponta para o objeto que recebe o retorno.



Mensagem de Guarda

- São usadas para modelar condições.
- Úteis quando é necessário restringir o fluxo de uma mensagem de acordo com uma condição.
- Representada por uma elipse ao redor da seta de mensagem indicando a condição.



Exemplos de Mensagens

71

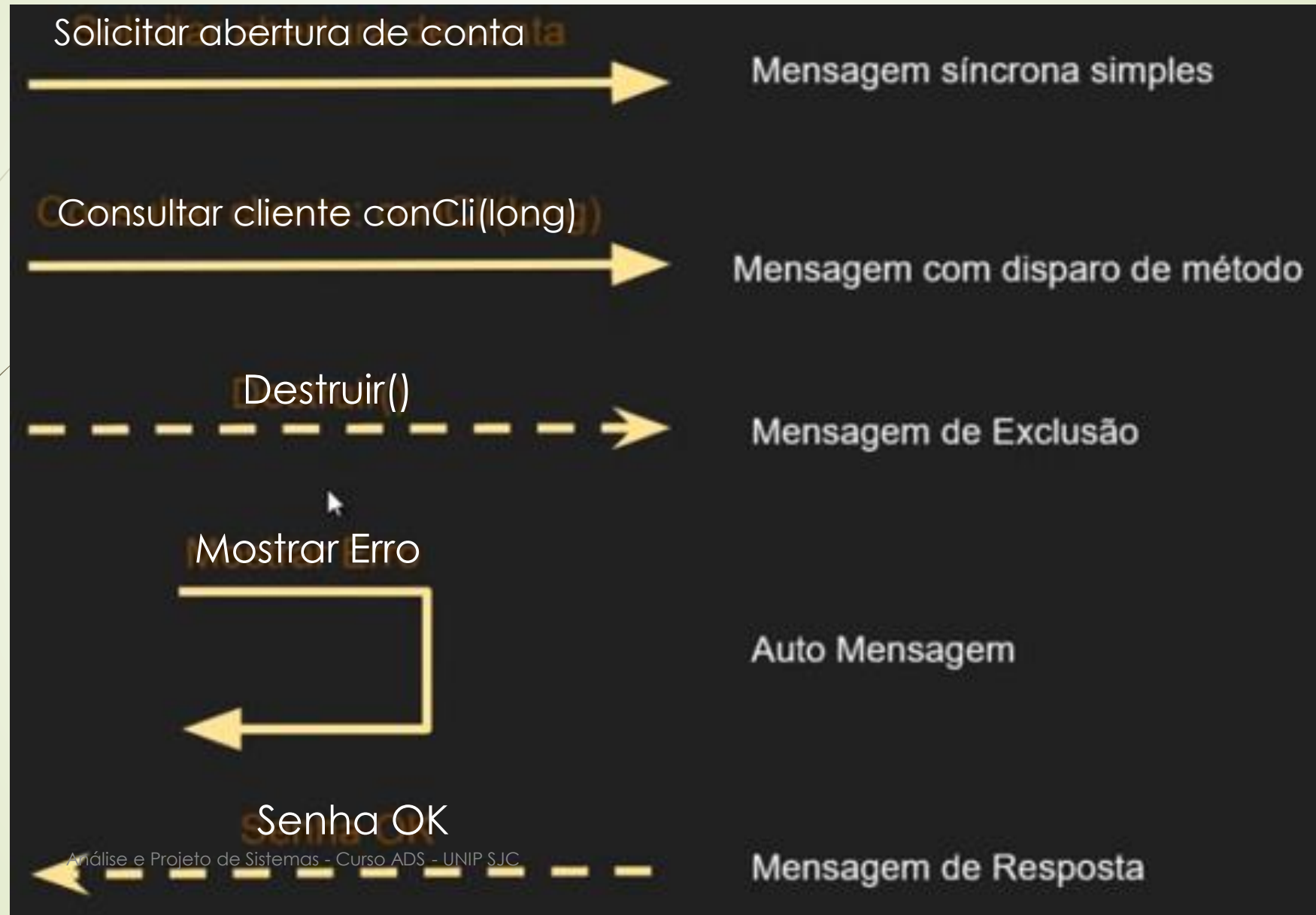
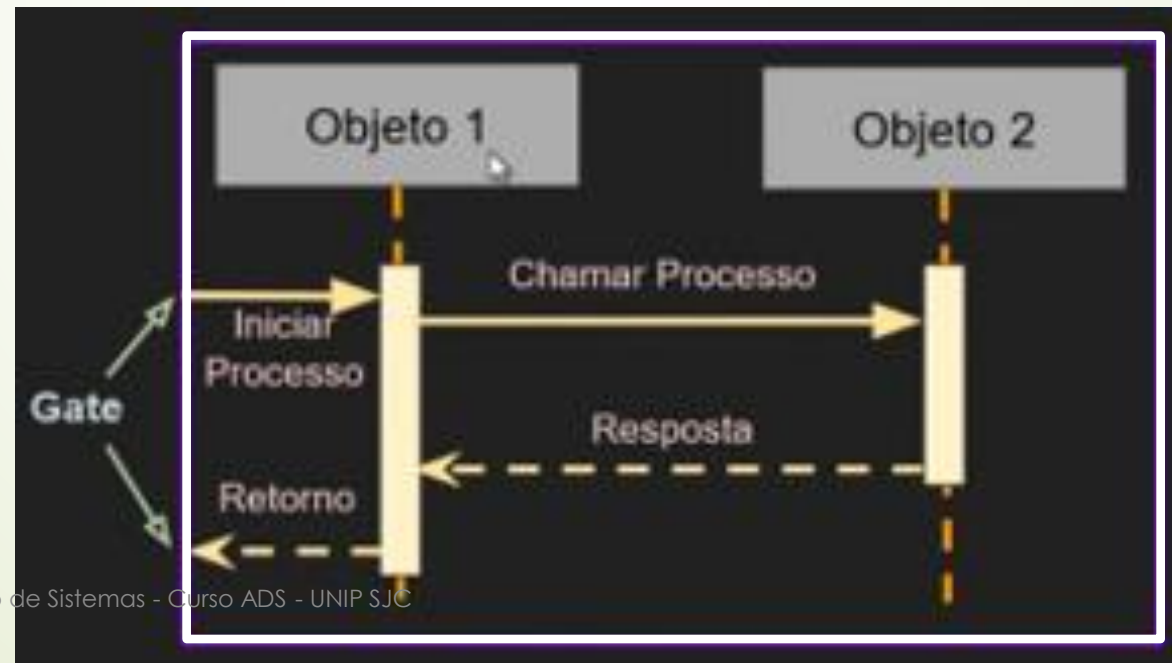


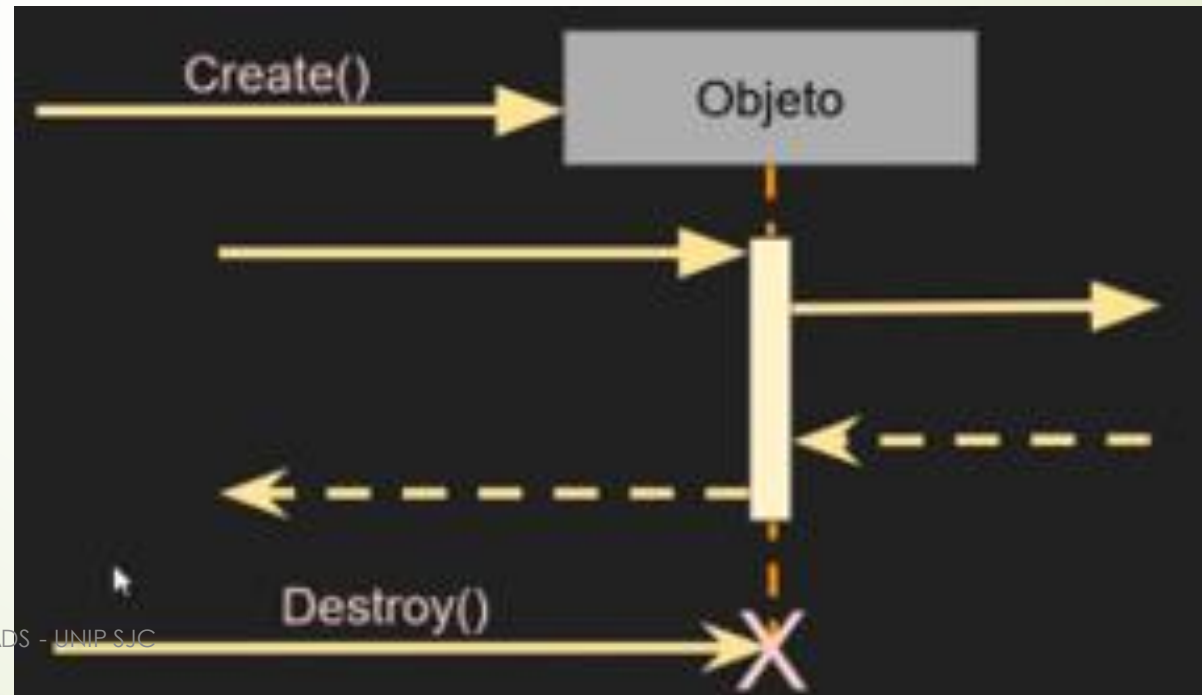
Diagrama de Sequência: Gate

- Um Gate (Portão) é o final de uma mensagem, é um ponto de conexão entre uma mensagem que esteja dentro de um fragmento de interação com outra que esteja fora.
- São representados como pontos de conexão de mensagem no quadro em si.



Tempo de vida de um objeto

- Se refere ao tempo em que o objeto existe, independente de estar realizando algum processo ou não. Quando um objeto não é mais necessário no sistema, seu tempo de vida pode expirar, e o objeto pode ser removido com uma mensagem de exclusão (Destroy())



Alguns tipos de operadores de Interação

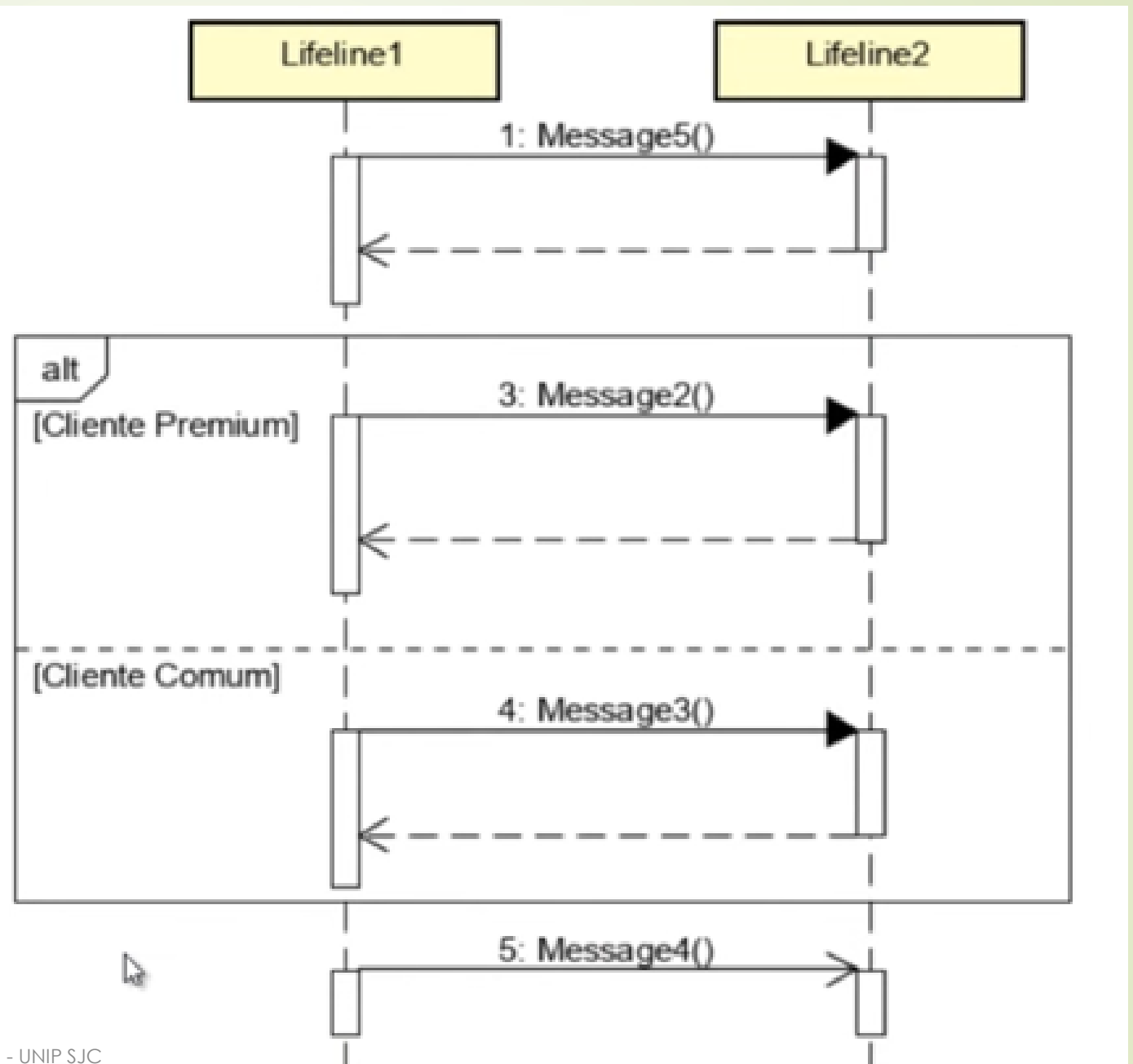
- Execução opcional (um ou nada): **opt**
- Execução condicional (um ou outro): **alt**
- Execução paralela: **par**
- Execução de loop (iteração): **loop**
- Ordem Sequencial Estrita: **strict**
- Interação inválida: **neg**
- Interação com outro diagrama: **ref**
- Quebra de execução (exceção): **break**
- Negação: **neg**

Fragmentos Combinados e Controle Estruturado

- Muitas vezes precisamos mostrar condicionais e loops, ou ainda a execução concorrente de várias sequências.
- Para isso usamos **Operadores de Controle Estruturado** (ou **Operadores de Interação**).
- Um Fragmento combinado define uma combinação de fragmentos de interação. Definido por um operador de interação e operandos de interação.
- É uma região retangular no diagrama, com uma **tag** que informa o tipo de operador. Aplica-se às linhas de vida que atravessam seu corpo (**representado por um retângulo com operador de interação**).

76

Exemplo de
fragmento de
decisão com o
operador de
fluxo alternativo:
alt



Como criar um Diagrama de Sequência

- 1. Selecionar um Caso de Uso no diagrama de casos de uso
- 2. Escrever uma descrição detalhada do que o caso de uso faz (sua narrativa)
- 3. Identificar os atores e objetos que interagem
- 4. Identificar as mensagens que os objetos trocam entre si
- 5. Determinar a sequência das mensagens
- 6. Determinar condições especiais (loops, fluxos alternativos, etc.)
- 7. Desenhar o diagrama

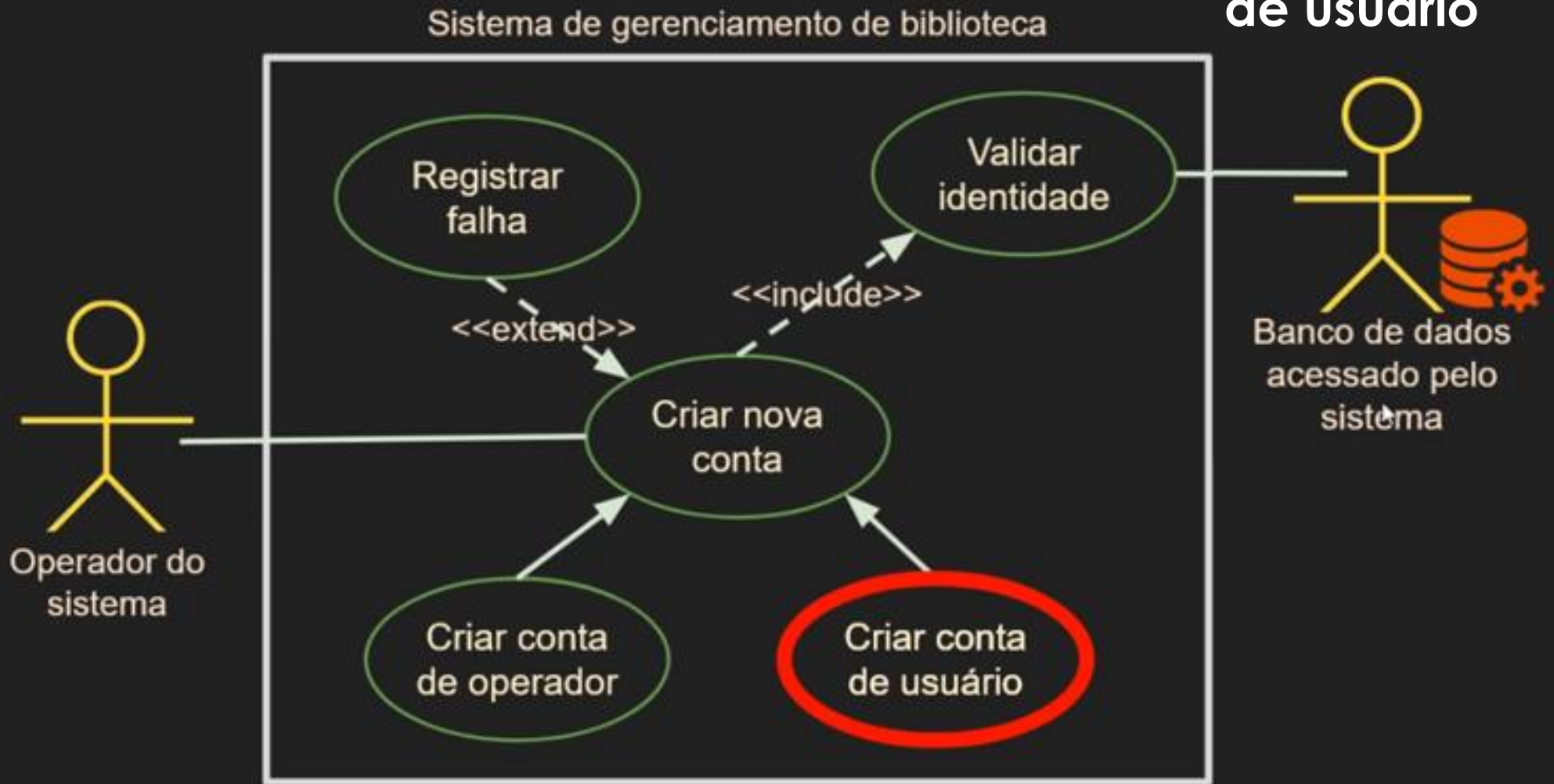
Sistema de Biblioteca

Exemplo de diagrama da Caso de Uso: **Sistema de Biblioteca.**

- Sistema permite:
 - cadastrar novos usuários (público em geral e operadores do sistema)
 - Realizar empréstimos de livros
 - Calcular multa por atraso na devolução dos livros
 - Cadastrar novos livros no acervo

Representado pelo diagrama de caso de uso a seguir.

Diagrama de Casos de uso (fragmento) > Criar conta de usuário



Identificar objetos e atores envolvidos

- Operador do sistema (bibliotecário)
- Sistema de Gerenciamento da Biblioteca
- Banco de dados do sistema



Descrição detalhada do caso de uso

- 1. Operador requisita ao sistema a criação de um novo usuário
- 2. Operador seleciona o tipo de conta de usuário
- 3. Operador fornece os dados do usuário
- 4. Sistema verifica os dados fornecidos no banco de dados
- 5. Nova conta de usuário é criada e gravada no BD
- 6. Sistema confirma a criação da conta

Etapa inicial da criação do diagrama

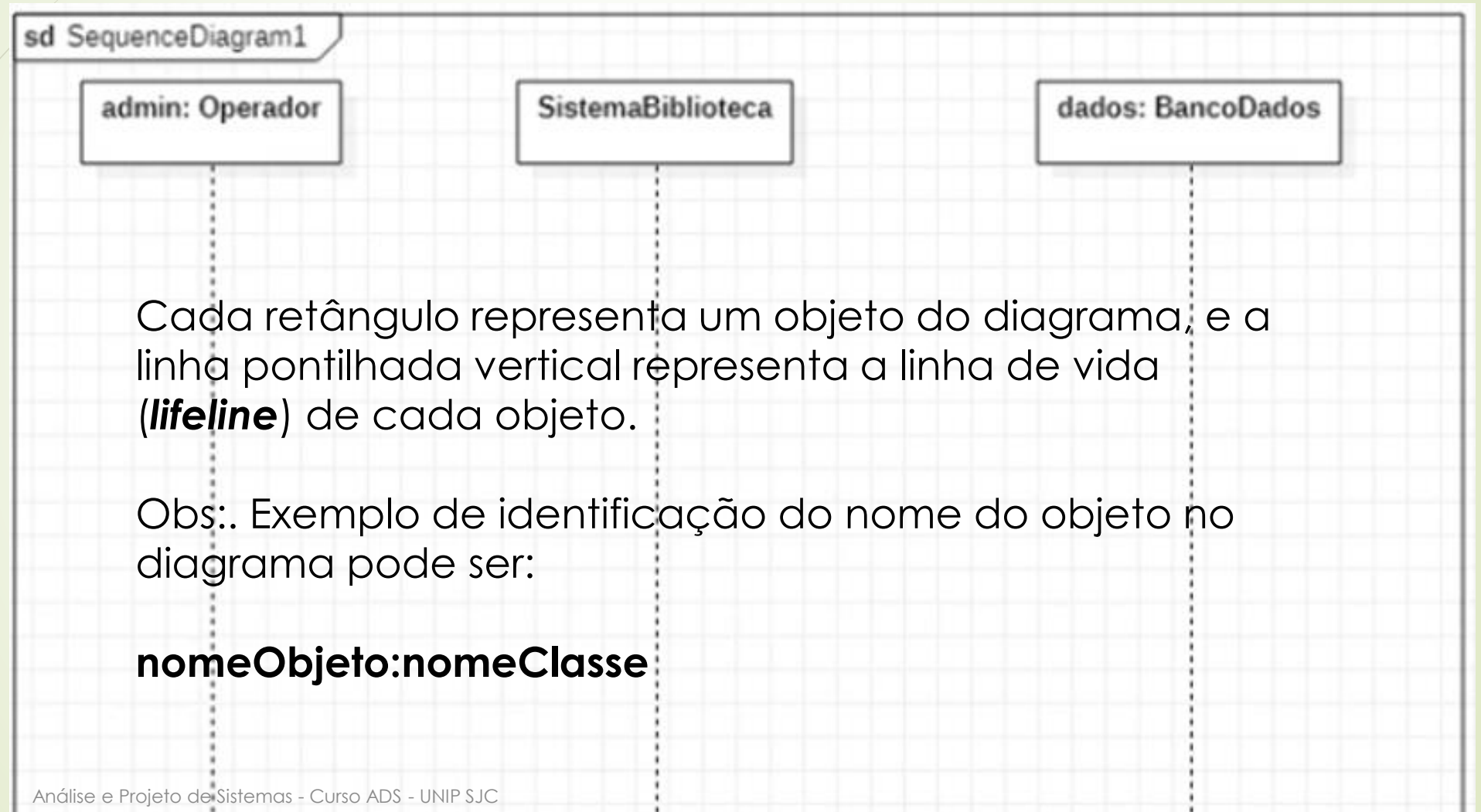
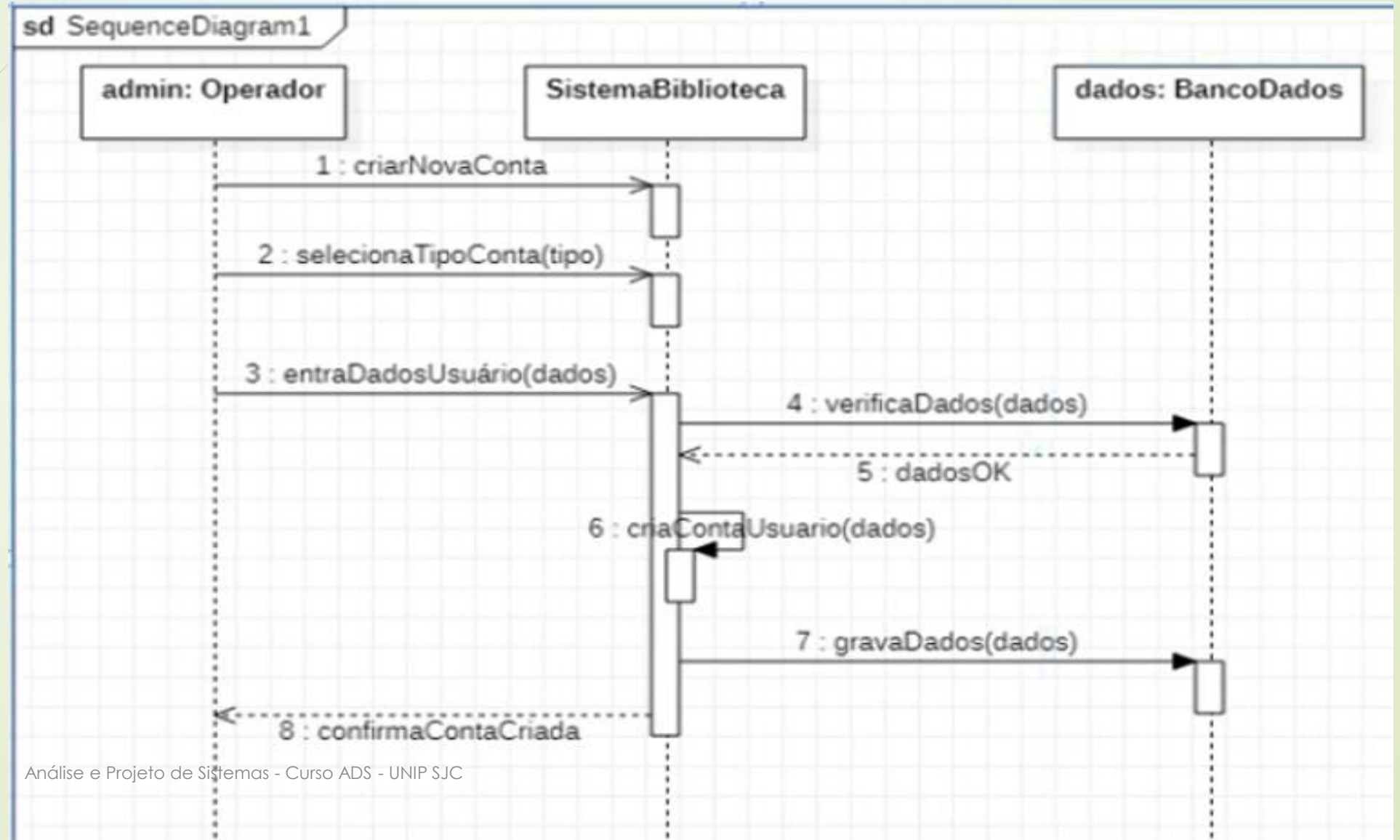
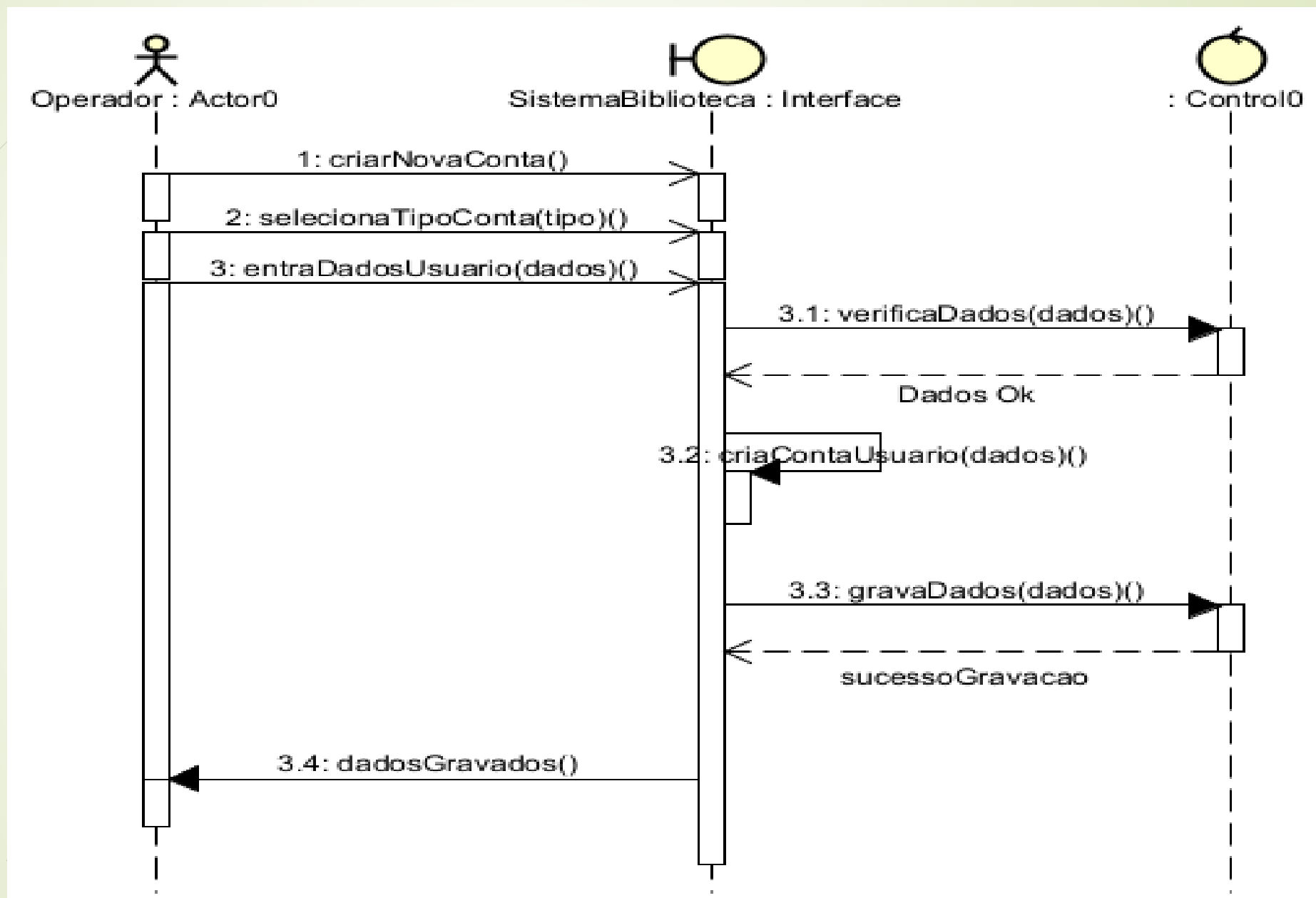


Diagrama de sequência para o caso de uso “criar conta de usuário”



Mesmo diagrama no Astah “criar conta de usuário”

84



Estrutura de Nomeação de Objetos no Diagrama de Sequência UML

- No diagrama de sequência UML, os objetos são representados por **linhas de vida** (*lifelines*), que indicam sua existência ao longo do tempo. A estrutura de nomeação desses objetos segue um padrão específico para facilitar a identificação e compreensão das interações no sistema.
- A seguir, estão os principais aspectos dessa estrutura:

Estrutura de Nomeação de Objetos no Diagrama de Sequência UML

➤ 1. Formato Geral:

O nome de um objeto é geralmente escrito no formato:

nomeObjeto : NomeClasse

- **nomeObjeto**: Representa a instância específica do objeto (opcional).
- **NomeClasse**: Indica a classe à qual o objeto pertence (obrigatório).

Estrutura de Nomeação de Objetos no Diagrama de Sequência UML

- Por exemplo:
- usuario : Usuario → Um objeto chamado "usuario" da classe "Usuario".
- : Produto → Um objeto anônimo da classe "Produto".
- **2. Objeto Anônimo:**
Quando o nome da instância não é relevante ou necessário, apenas o nome da classe é especificado, precedido por dois pontos:
 - Exemplo: : Pedido.

Estrutura de Nomeação de Objetos no Diagrama de Sequência UML

■ 3. Posicionamento no Diagrama:

- O nome do objeto é colocado **dentro de um retângulo** no topo da linha de vida.
- A linha de vida se estende verticalmente a partir do retângulo, representando o tempo de existência do objeto.

■ 4. Diferenciação de Objetos:

- Para evitar ambiguidades, objetos com o mesmo tipo de classe podem ter nomes de instância diferentes.
- Exemplo: cliente1 : Cliente e cliente2 : Cliente.

Estrutura de Nomeação de Objetos no Diagrama de Sequência UML

➤ 5. Mensagens e Interações:

- Os objetos nomeados trocam mensagens entre si, representadas por setas horizontais. O nome do objeto ajuda a identificar claramente quem envia e quem recebe cada mensagem.
- Essa estrutura de nomeação é essencial para manter o diagrama claro e compreensível, especialmente em sistemas complexos com múltiplos objetos interagindo.



OO – Padrões de projetos

APS – Análise e Projetos de Sistemas

Prof. Francisco Martins

DESIGN PATTERNS: Breve introdução aos padrões de projeto



DESIGN PATTERNS: Breve introdução aos padrões de projeto

- Como na matemática, física ou química, nas quais foram desenvolvidas teorias para se resolver determinadas questões, a programação também passou pelo mesmo processo.
- No passado foram desenvolvidas fórmulas, que hoje chamamos de *design patterns* (Padrões de projeto), que no final, se resumem a soluções típicas para problemas comuns em um projeto de software. Elas podem ser aplicadas independentemente da linguagem.

DESIGN PATTERNS: O que é padrão de projeto?

- ▶ São como plantas pré-projetadas de uma construção, que você pode alterar para se adequar melhor na resolução de um problema recorrente em seu código. O que diferencia os padrões de projeto das funções e bibliotecas é que **você não pode simplesmente copiá-los direto para seu programa**, já que eles não são um pedaço de código, **mas sim um conceito** que serve como uma solução.

DESIGN PATTERNS: O que é padrão de projeto?

- Assim, para se implementar um padrão de projeto você deve seguir o conceito dos padrões escolhidos (dentre todos os existentes) e ajustá-lo ao problema que deseja resolver. Isso dependerá das características do projeto. Dessa forma, fazemos uma implementação que se encaixe exatamente à demanda da nossa aplicação.

DESIGN PATTERNS: **Benefícios**

- Um bom conselho que costumamos dar e receber na área de exatas é para **evitarmos de reinventar a roda** e ele consegue se encaixar bem nessa situação, pois não faz sentido gastar tempo analisando e criando quando já existe um “guia” pronto para ser seguido.

DESIGN PATTERNS: A sigla SOLID

- O SOLID é uma sigla em inglês para cinco princípios de projeto que possuem o objetivo de fazer programas mais compreensíveis, flexíveis e sustentáveis.
- Não faz parte dos design patterns, mas por serem boas práticas que garantem diversos benefícios é bem comum de aprender ambos os temas ao mesmo tempo.

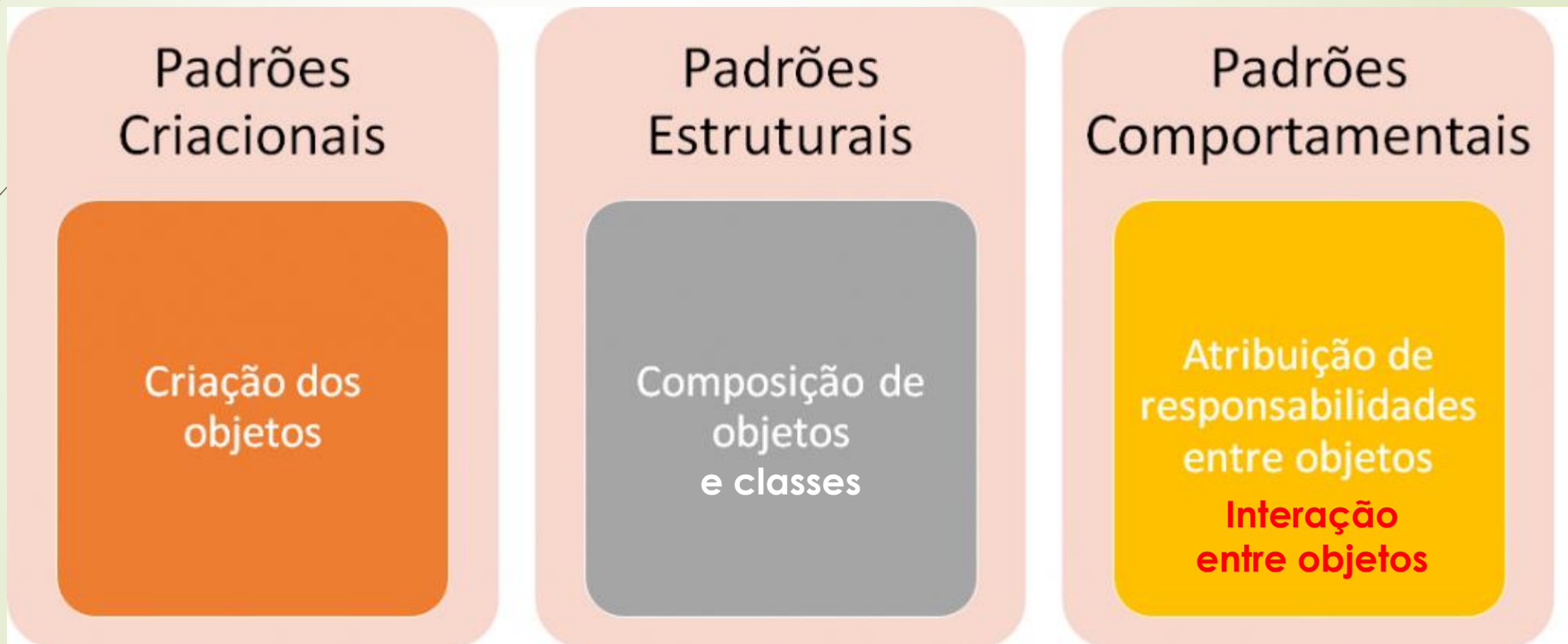
DESIGN PATTERNS: A sigla SOLID

- “S” Single Responsibility Principle (Princípio de responsabilidade única): **uma classe deve ter uma e apenas uma razão para mudar.**
- “O” Open-Closed Principle (Princípio aberto/fechado): **objetos devem estar disponíveis para extensão, mas fechados para modificação.**
- “L” Liskov Substitution Principle (Princípio de substituição de Liskov): **uma subclasse deve ser substituível por sua superclasse.**
- “I” Interface Segregation Principle (Princípio de segregação de interface): **uma classe não deve ser obrigada a implementar métodos e interfaces que não serão utilizados.**
- “D” Dependency Inversion Principle (Princípio de inversão de dependência): **dependa de abstrações e não de implementações.**

DESIGN PATTERNS: Principais padrões

- Os três principais padrões de projeto definidos pelo livro “**Design Patterns: Elements of Reusable Object-Oriented Software**” de 1994, escrito por GOF (Gang of Four: Rich Gamma, Richard Helm, Ralph Johnson e John Vlissides) são os **padrões criacionais, estruturais e comportamentais**. Esses padrões foram divididos e agrupados de acordo com a natureza do problema que eles solucionam.

Agrupamento por natureza



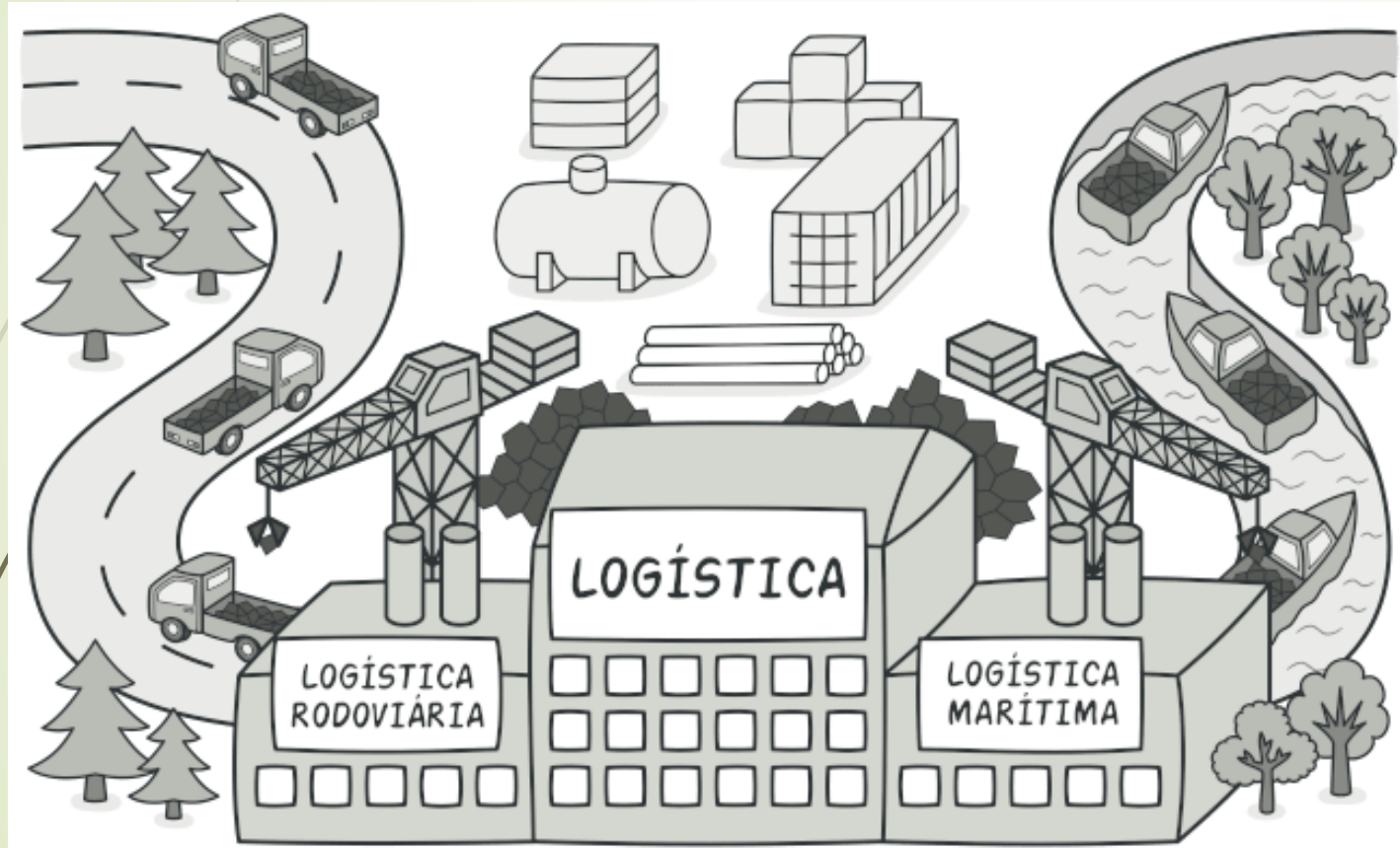
Agrupamento por natureza

		Propósito		
		Criação	Estrutura	Comportamento
Escopo	Objeto	Builder Abstract Factory Prototype Singleton Factory Method	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor Interpreter Template Method

DESIGN PATTERNS: 1 - Criacionais

- 1. Padrões criacionais: estes padrões oferecem diversas alternativas de criação de objetos, o que aumenta a flexibilidade e a reutilização de código. Alguns dos principais padrões desse tipo são:
- **Factory Method**
- **Abstract Factory**
- **Builder**
- **Singleton**

Padrão Factory Method



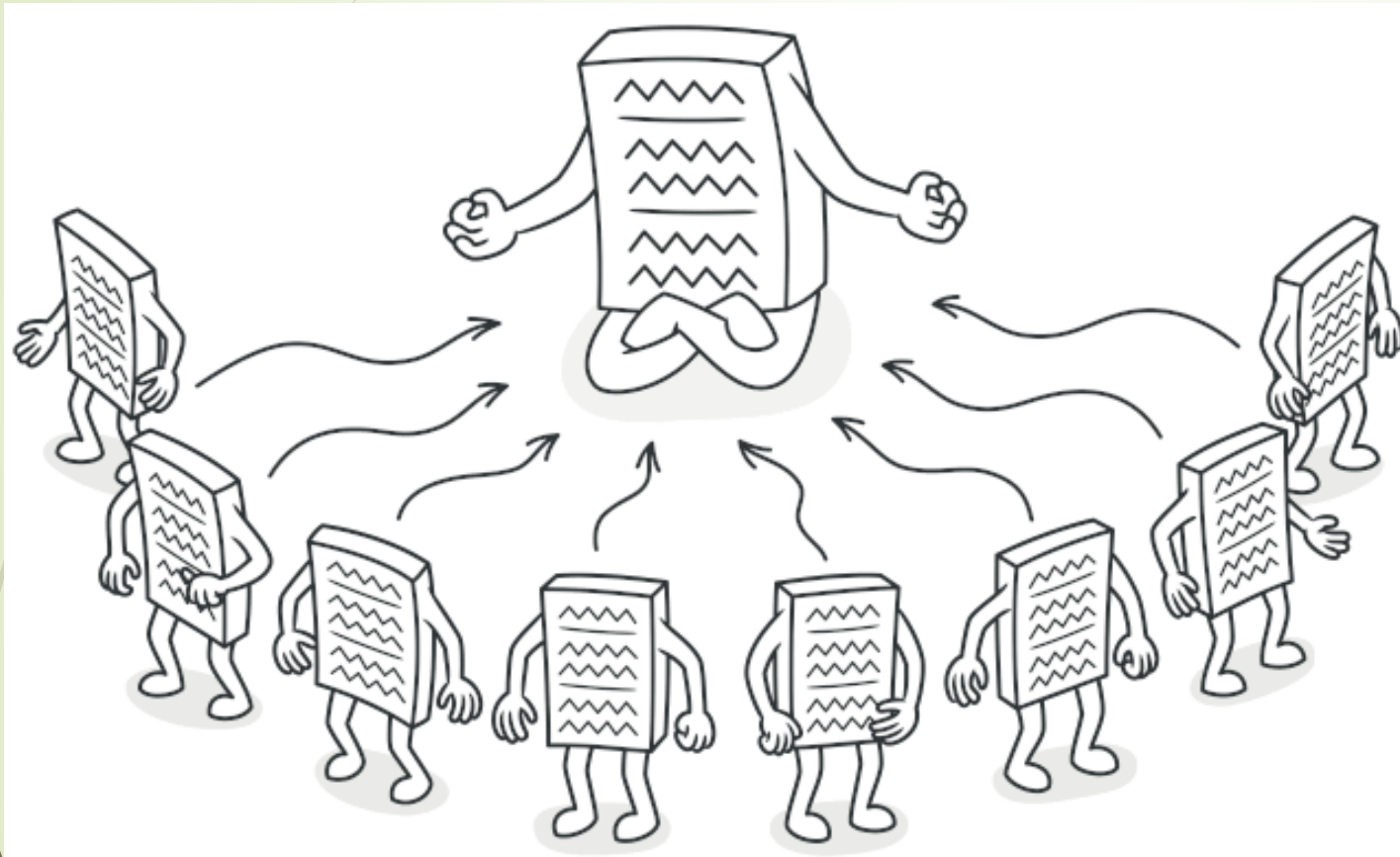
- O **Factory Method** é um padrão criacional de projeto que fornece uma interface para criar objetos em uma superclasse, mas permite que as subclasses alterem o tipo de objetos que serão criados.

Padrão Abstract Factory



- O **Abstract Factory** é um padrão de projeto criacional que permite que você produza famílias de objetos relacionados sem ter que especificar suas classes concretas.

Padrão Singleton

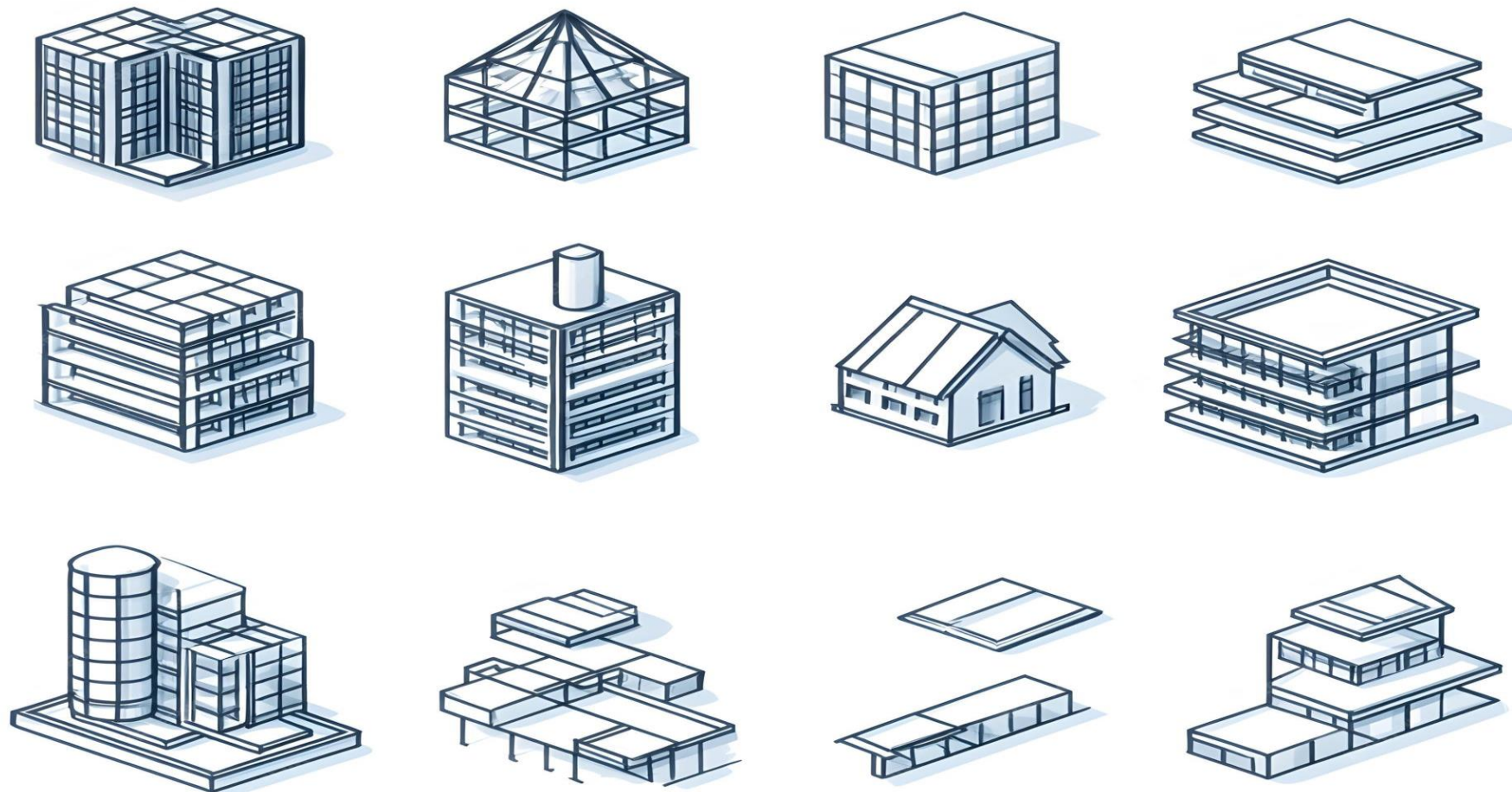


- O **Singleton** é um padrão de projeto criacional que permite a você garantir que uma classe tenha apenas uma instância, enquanto provê um ponto de acesso global para essa instância.

DESIGN PATTERNS: 2 - Estruturais

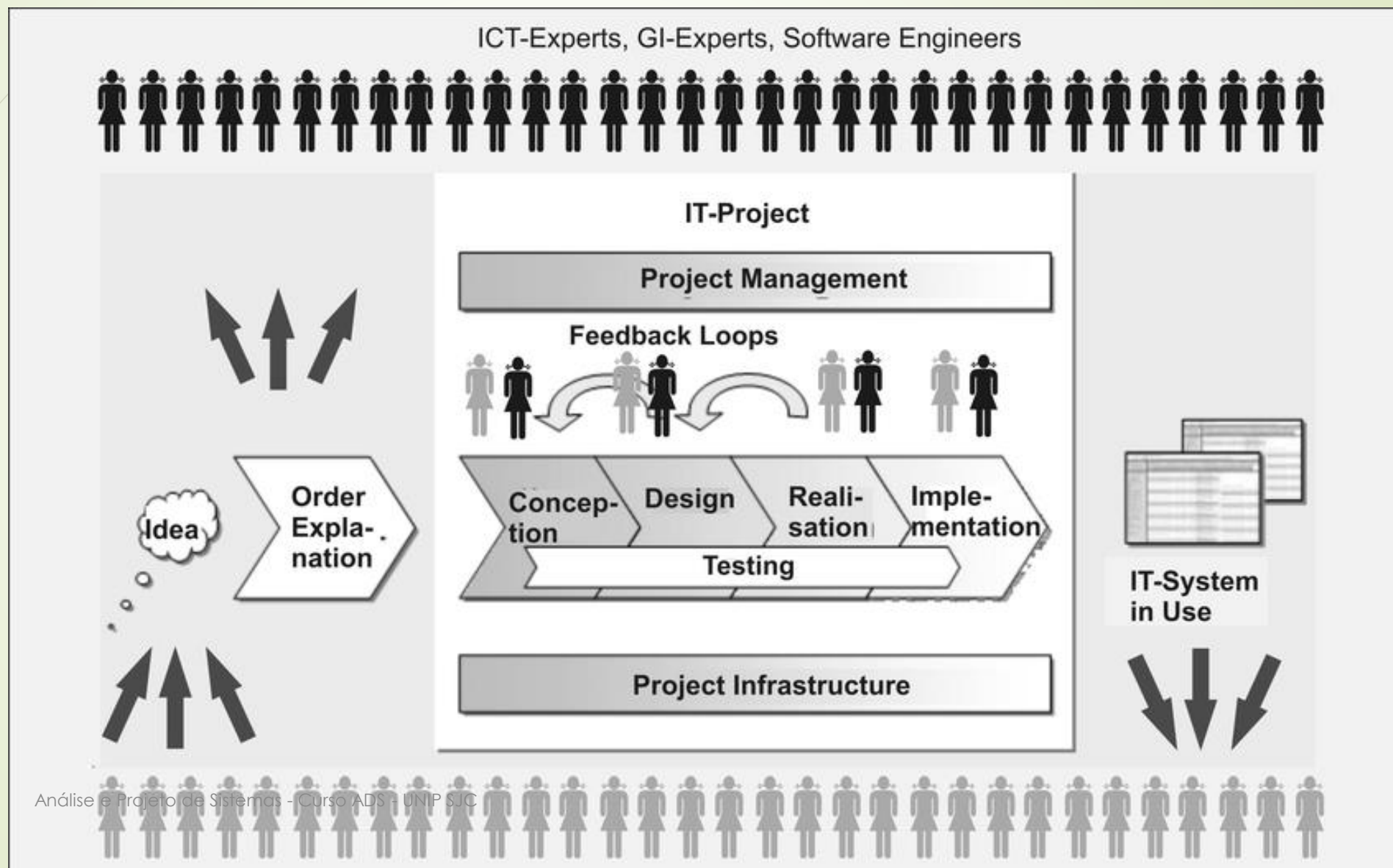
- 2. Padrões estruturais: Nos mostram como montar objetos e classes em estruturas maiores, sem perder a eficiência e flexibilidade. Alguns dos principais padrões desse tipo são:
- Adapter
- Bridge
- **Composite**

Estrutura em blocos de montagem (Lego)



Estrutura do ciclo de vida do projeto

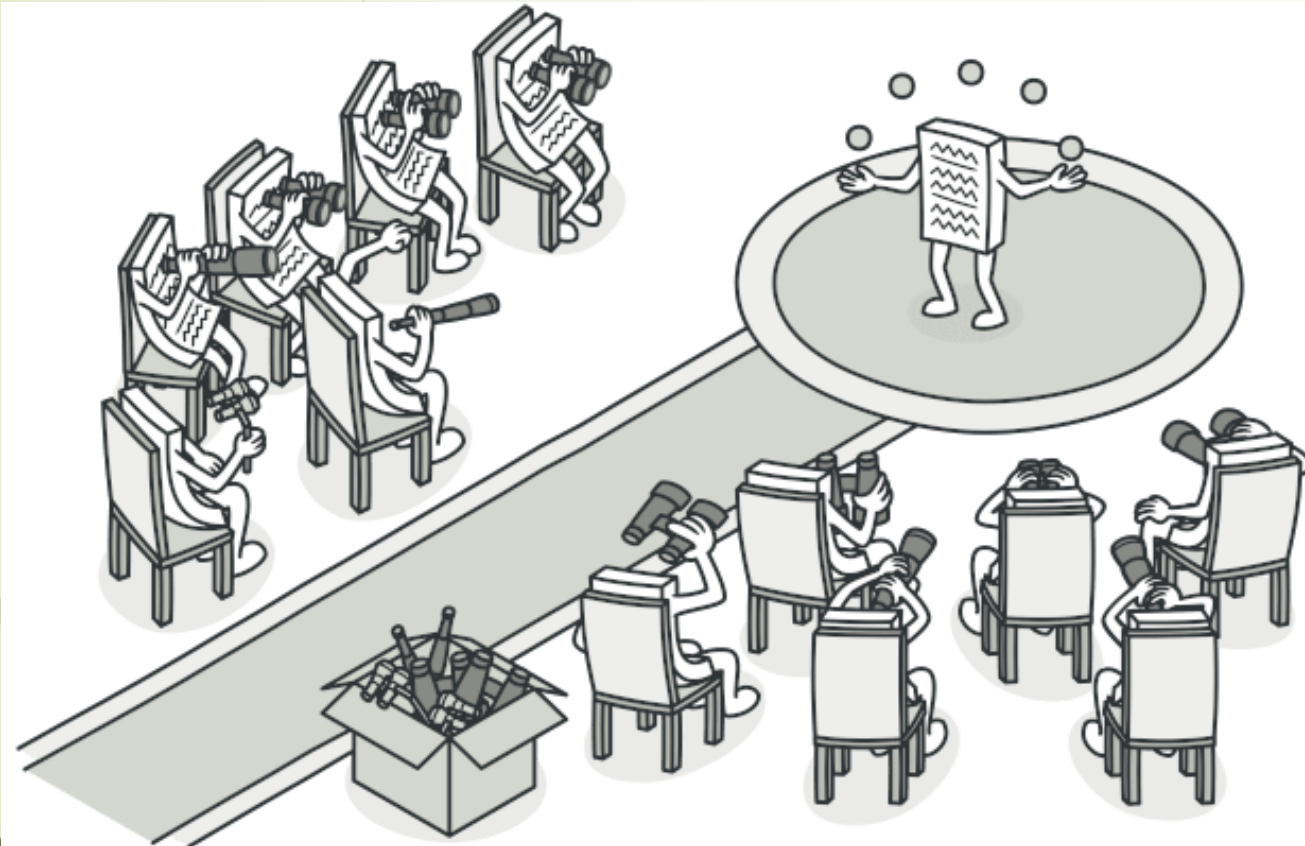
107



DESIGN PATTERNS: 3 - Comportamentais

- 3. Padrões comportamentais: Nos ajudam a trabalhar melhor com os algoritmos e com a delegação de responsabilidades entre os objetos. Os padrões que se destacam nesse tipo são:
- Chain of Responsibility
- Command
- Interpreter
- **Observer / Strategy**

Padrão Observer



- O **Observer** é um padrão de projeto comportamental que permite que você defina um mecanismo de assinatura para notificar múltiplos objetos sobre quaisquer eventos que aconteçam com o objeto que eles estão observando.

Padrão Strategy



- O **Strategy** é um padrão de projeto comportamental que permite que você defina uma família de algoritmos, coloque-os em classes separadas, e faça os objetos deles intercambiáveis.

DESIGN PATTERNS: Conclusão

- Conhecer e utilizar **design patterns** é algo de muita importância no desenvolvimento profissional de qualquer software.
- O uso desses padrões nos auxilia nos desafios mais recorrentes durante o desenvolvimento, além de nos oferecer uma maior produtividade e garantir que você saia na frente como profissional.
- Portanto, é importante ser um estudioso em padrões de projeto.